

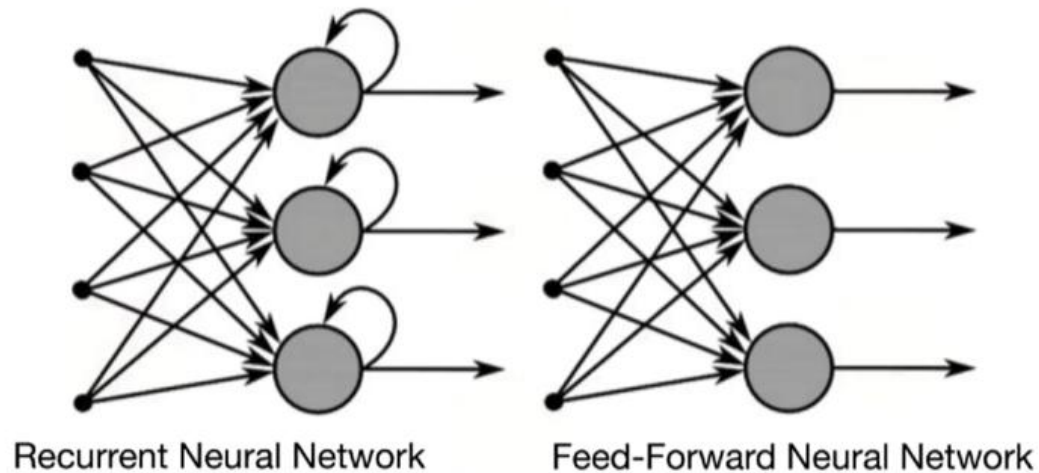
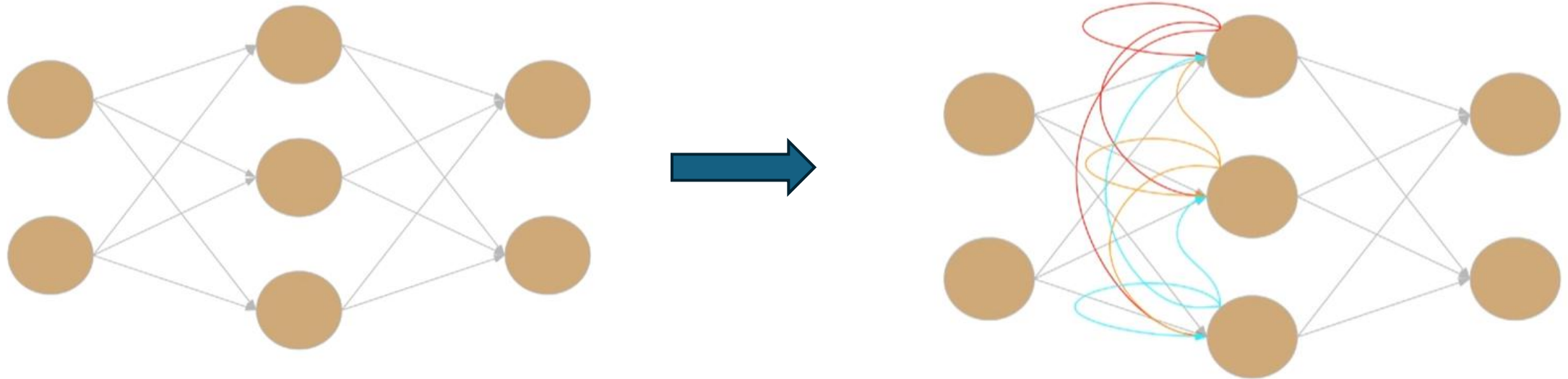
Recurrent Neural Networks (RNNs) and Long Short Term Memory Networks (LSTMs)

RNNs

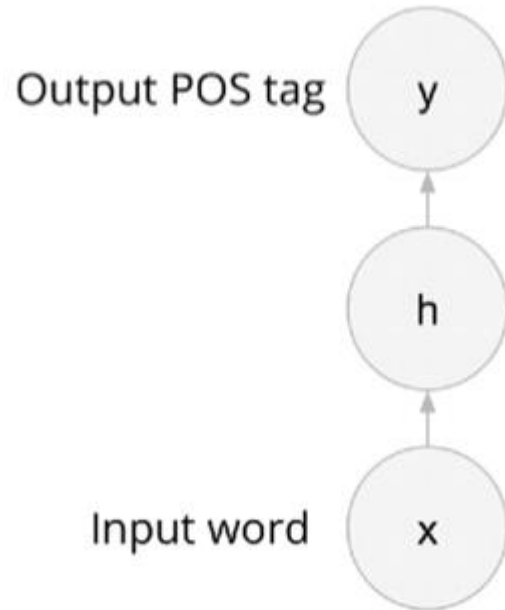
LSTMs

Recurrent Neural Networks (RNNs)

Recurrent Neural Networks (RNNs)



Recurrent Neural Networks (RNNs)



8 Parts of Speech

Adjectives	Adverbs
Conjunctions	Interjections
Nouns	Prepositions
Pronouns	Verbs

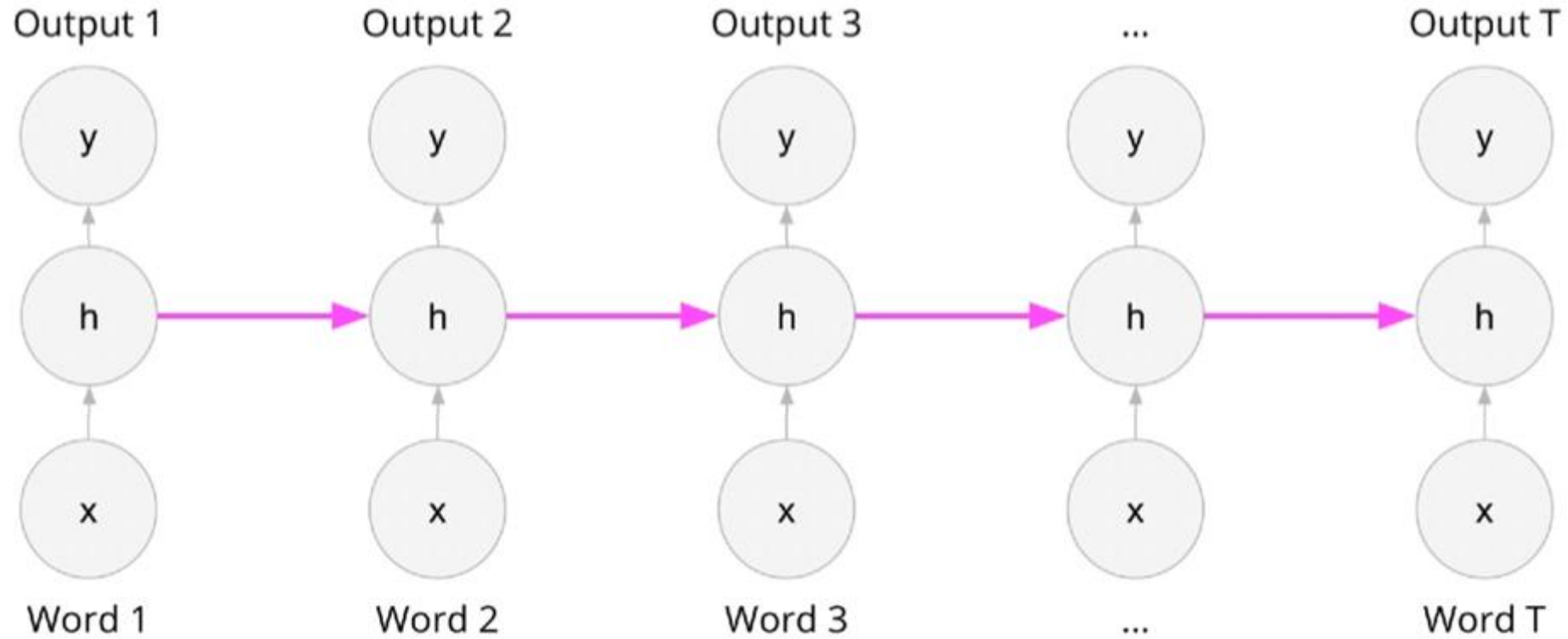
Why dont use ANNs?

Problem with ANN

“Your bank has requested your password.”
“Let’s take a walk on the river bank.”
“Would you bank on it?”
“I bank with Bank of America.”

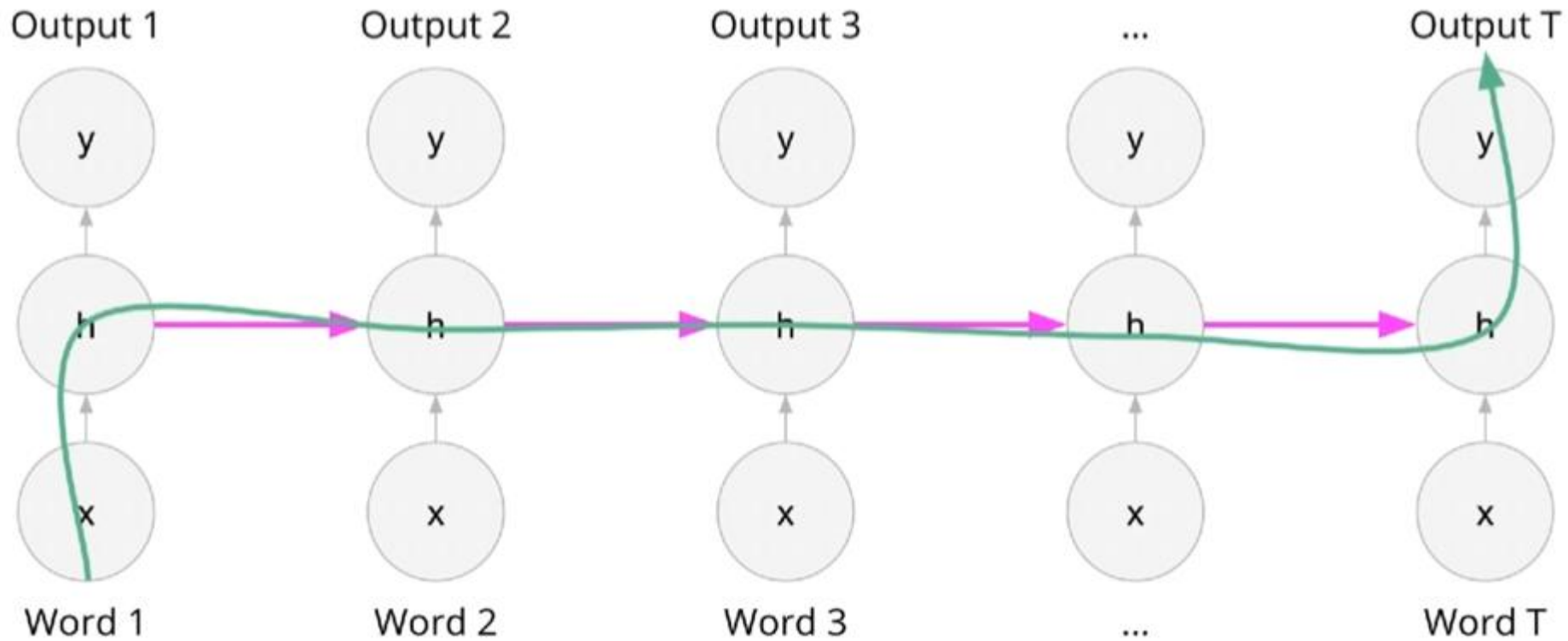
- Sometimes there can be multiple possible answers, which can only be disambiguated with context
- E.g. “bank” has several meanings
 - Financial institution (maybe higher chance of spam)
 - “River bank” (maybe lower chance of spam)
 - As a verb (and still has multiple meanings!)

Recurrent Neural Networks (RNNs)



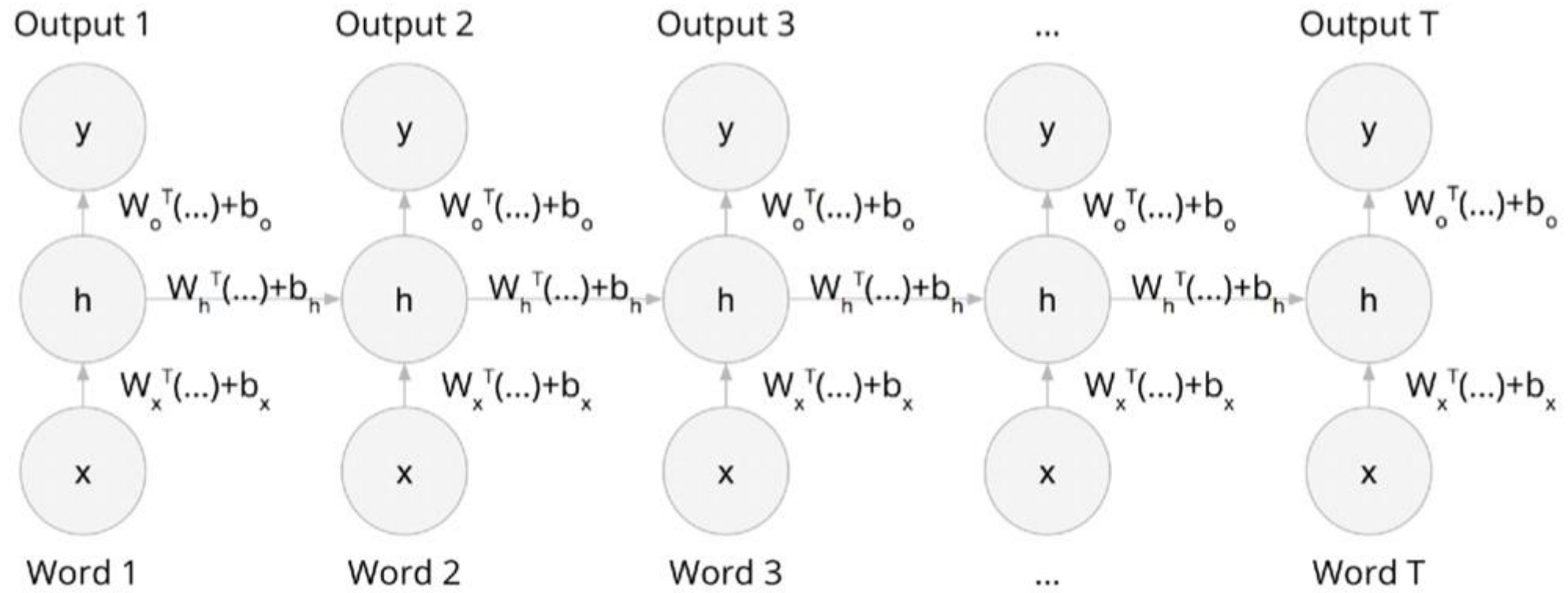
Recurrent Neural Networks (RNNs)

Notice: there is a pathway from all previous words up to the current output



Recurrent?

Note: activation functions omitted for simplicity

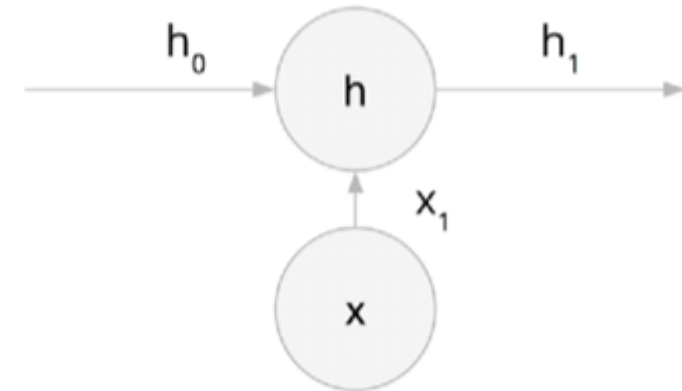


Recurrent?

- Consider the first step of the RNN
- Typically we assume the first hidden state vector is zero

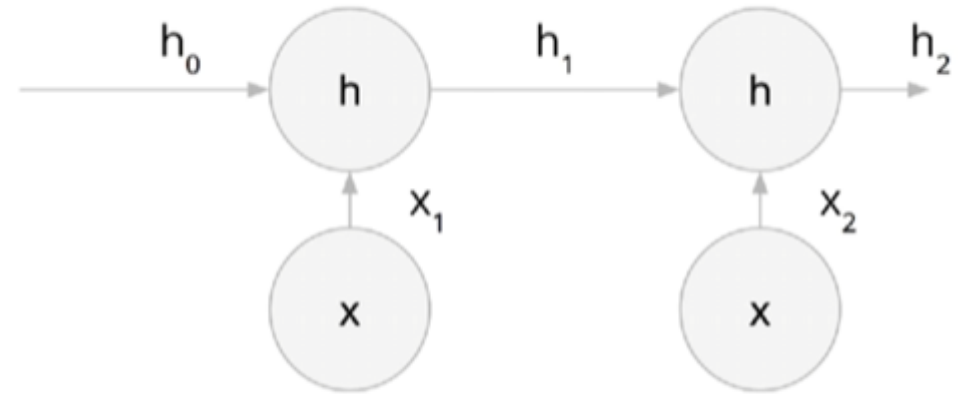
Any typical activation function

$$h_1 = \sigma(W_h^T h_0 + b_h + W_x^T x_1 + b_x)$$



Recurrent?

- Consider the second step of the RNN

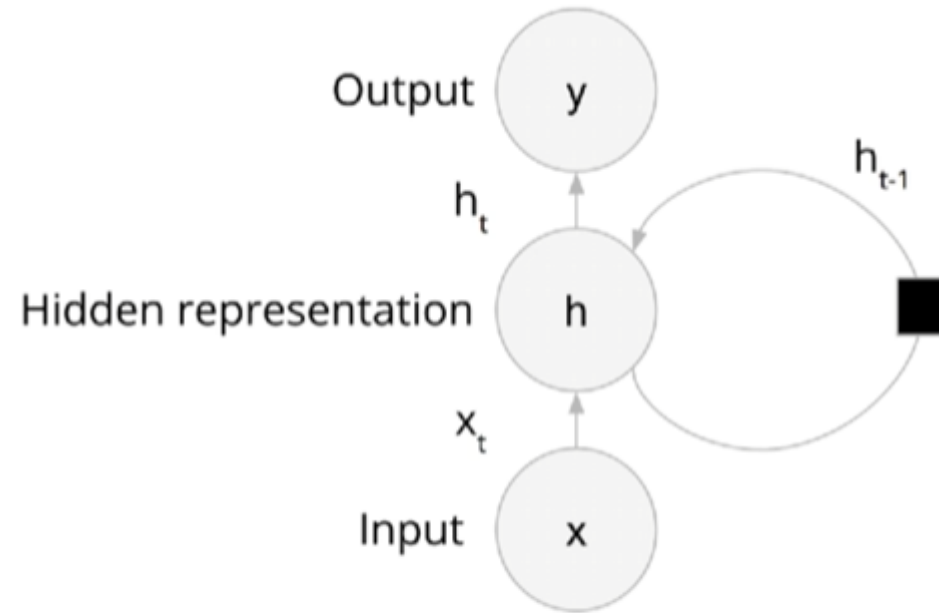


Any typical activation function

$$h_2 = \sigma(W_h^T h_1 + b_h + W_x^T x_2 + b_x)$$

Recurrent?

- Every step is the same!



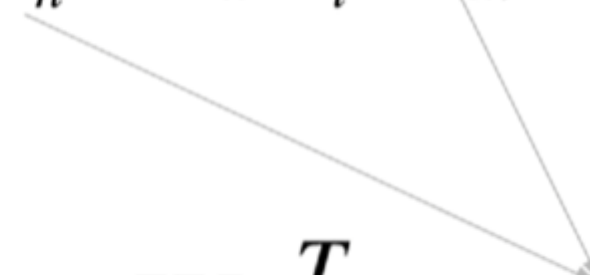
$$h_t = \sigma(W_h^T h_{t-1} + b_h + W_x^T x_t + b_x)$$

$$\textit{Fibonacci} : x_n = x_{n-1} + x_{n-2}$$

Alternative Representations

- Having multiple bias terms is redundant

$$h_t = \sigma(W_h^T h_{t-1} + b_h + W_x^T x_t + b_x)$$



The diagram consists of two thin grey arrows. One arrow originates from the b_h term in the top equation and points to the b_h term in the bottom equation. The other arrow originates from the b_x term in the top equation and also points to the b_h term in the bottom equation, illustrating that the two bias terms are combined into a single one.

$$h_t = \sigma(W_h^T h_{t-1} + W_x^T x_t + b_h)$$

.

Alternative Representations

- Combine hidden vector / input vector, and hidden weight / input weight

$$h_t = \sigma(W_v^T \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix} + b_h), \text{ where } W_v = \begin{bmatrix} W_h \\ W_x \end{bmatrix}$$

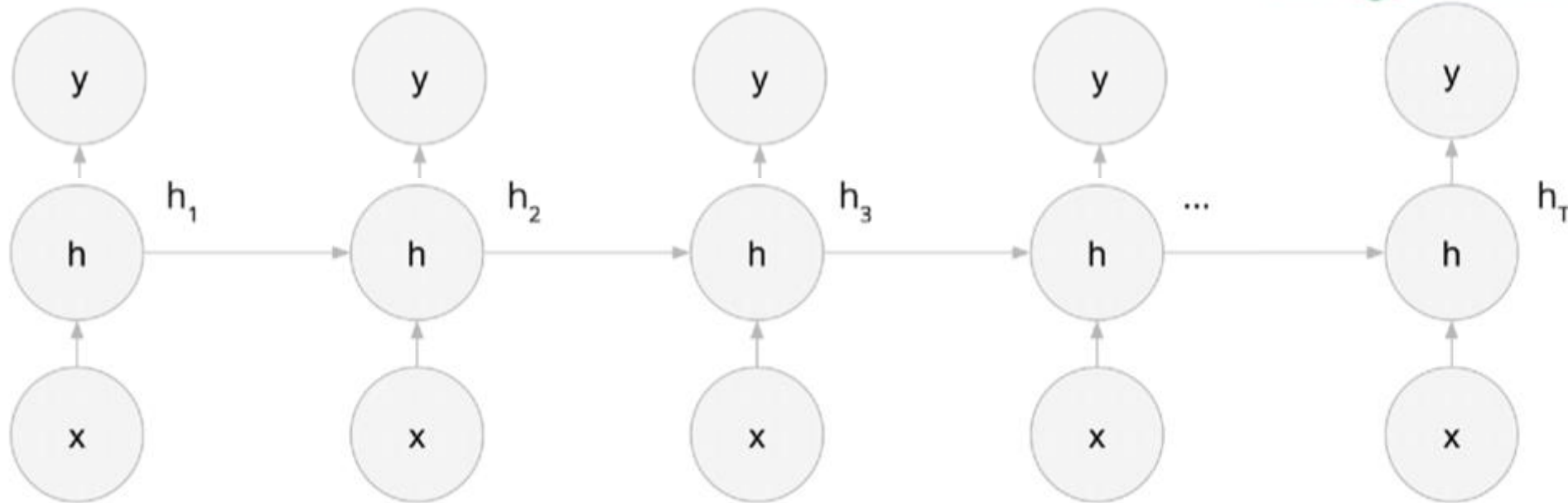
Size D + M vector

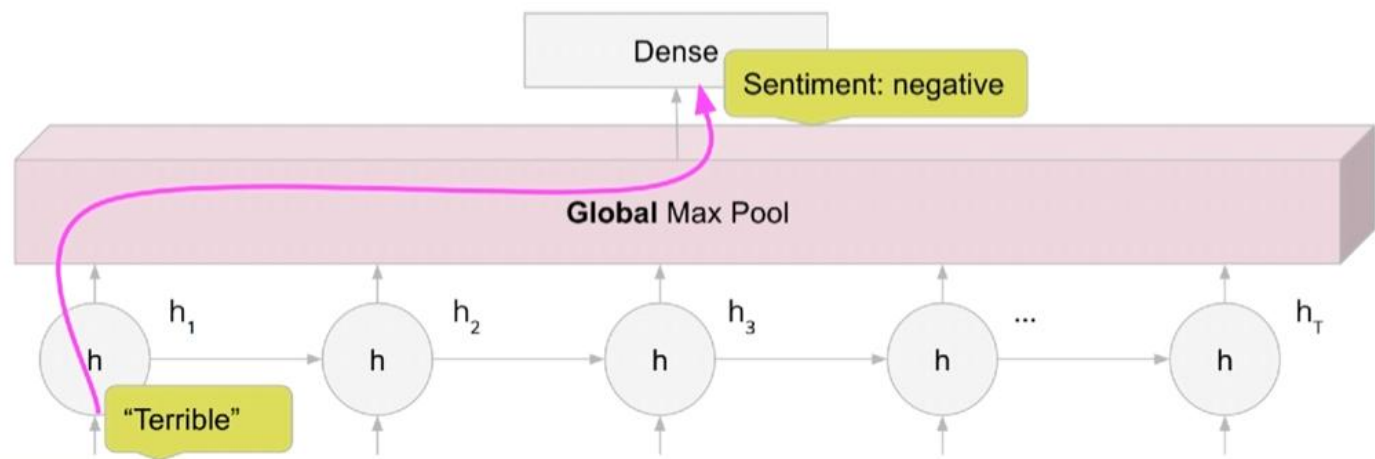
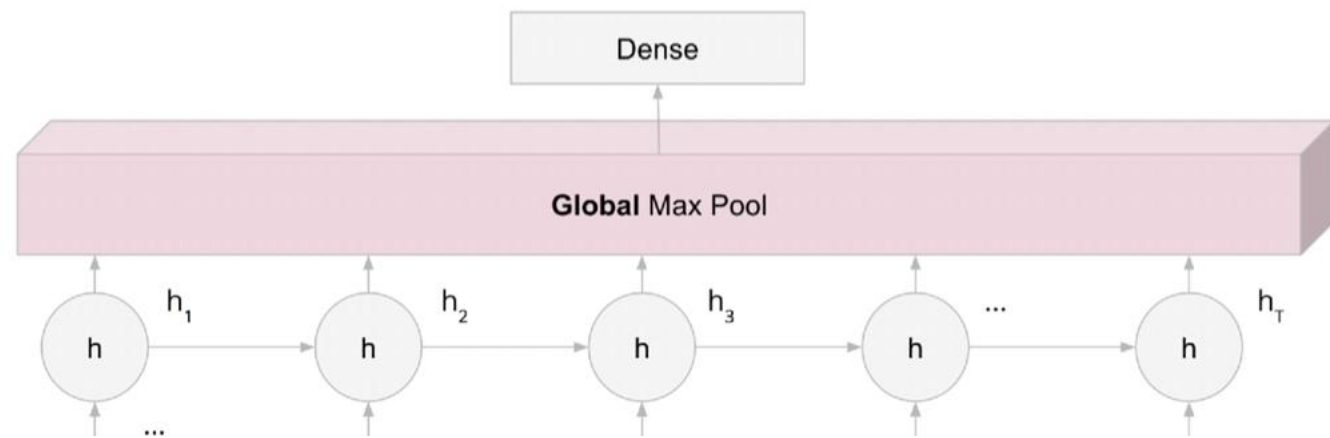
Size (D + M) x M matrix

$h(t)$?

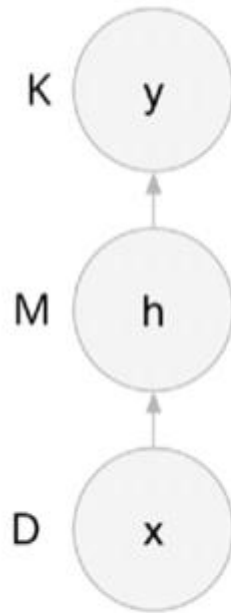
- $h(t)$ is the output of the RNN at each time step - what happens to it?

Only compute this
for many-to-one





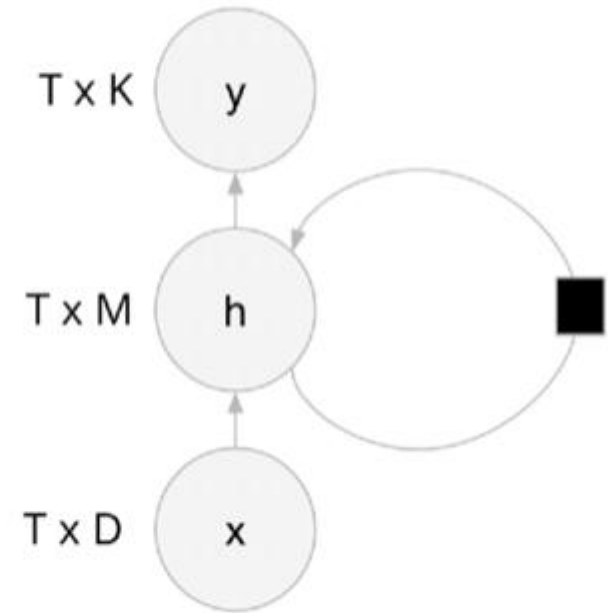
ANN vs RNN



For concreteness, you can pretend $K = 8$

8 Parts of Speech

Adjectives	Adverbs
Conjunctions	Interjections
Nouns	Prepositions
Pronouns	Verbs



Recurrent Neural Networks (RNNs)

- Language is an inherently **temporal phenomenon**.
 - Language is a **sequence of events over time**.
- This **temporal nature** is reflected in some language processing algorithms.
 - HMM part-of-speech tagging proceeds through the input a word at a time.
 - Text classification tasks don't have this temporal nature. They assume simultaneous access to all aspects of their input.
- The **feedforward networks** also assumed simultaneous access, although they also had a simple model for time.
 - Feedforward networks to language modeling look at a **fixed-size window of words**, and then sliding this window over the input.
- **Recurrent Neural Networks (RNNs)**, and their variants like **LSTMs** are **deep learning architectures** that offers an alternative way of representing time.

What is a Recurrent Neural Network (RNN)?

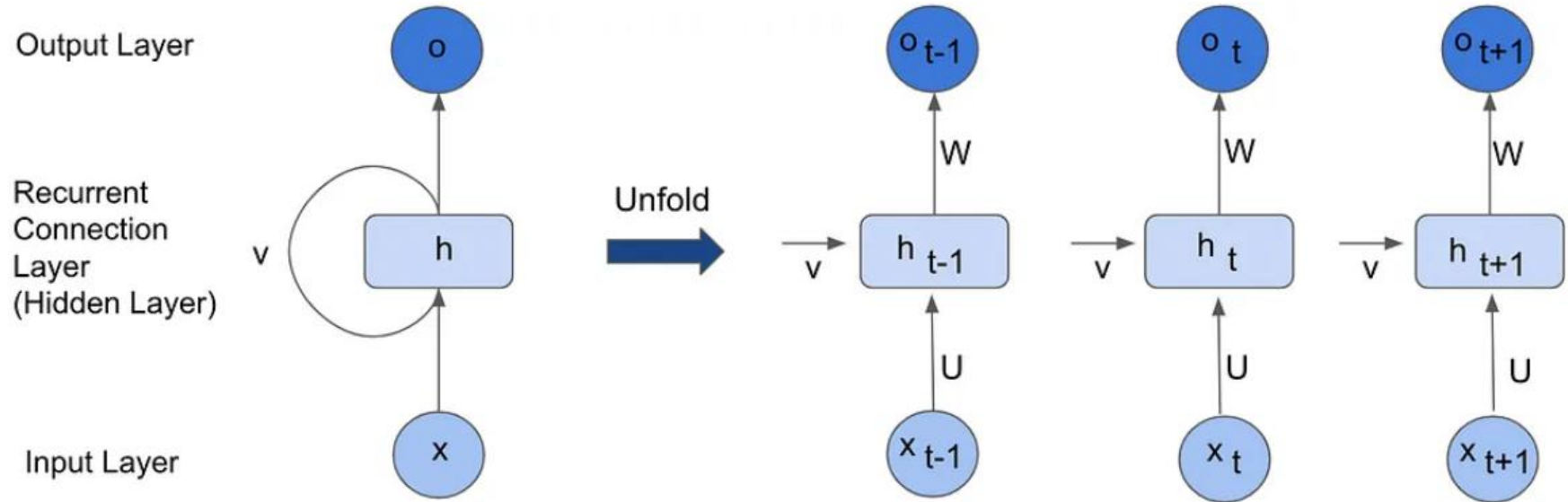
- RNN is a deep neural network trained on sequential or time series data to create a machine learning (ML) model that can make sequential predictions or conclusions based on sequential inputs.
- Recurrent Neural Networks introduce a mechanism where the output from one step is fed back as input to the next, allowing them to retain information from previous inputs. This design makes RNNs well-suited for tasks where context from earlier steps is essential, such as predicting the next word in a sentence.

Recurrent Neural Networks (RNNs)

- **RNNs** have a mechanism that deals directly with the sequential nature of language, allowing them to handle the temporal nature of language without the use of arbitrary fixed-sized windows.
- The **recurrent network** represents the prior context, in its **recurrent connections**, allowing the model's decision to depend on hundreds of words in the past.
- A **recurrent neural network (RNN)** is **any network that contains a cycle within its network connections**,
 - An input value of some unit is depends on its own earlier outputs as an input.

Key Components of RNNs

- Input Layer
- Hidden Layer
- Activation Function
- Output Layer
- Recurrent Connection



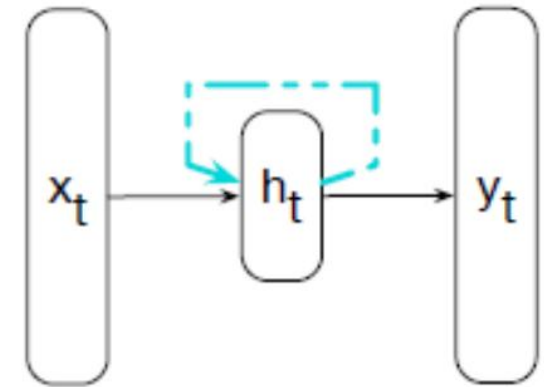
- **Input Layer:** This layer receives the initial element of the sequence data. For example, in a sentence, it might receive the first word as a vector representation.
- **Hidden Layer:** The heart of the RNN, the hidden layer contains a set of interconnected neurons. Each neuron processes the current input along with the information from the previous hidden layer's state. This “state” captures the network's memory of past inputs, allowing it to understand the current element in context.
- **Activation Function:** This function introduces non-linearity into the network, enabling it to learn complex patterns. It transforms the combined input from the current input layer and the previous hidden layer state before passing it on.

- **Output Layer:** The output layer generates the network's prediction based on the processed information. In a language model, it might predict the next word in the sequence.
- **Recurrent Connection:** A key distinction of RNNs is the recurrent connection within the hidden layer. This connection allows the network to pass the hidden state information (the network's memory) to the next time step. It's like passing a baton in a relay race, carrying information about previous inputs forward.

Recurrent Neural Networks (RNNs)

simple recurrent neural network

- **Simple recurrent neural network**
- The **hidden layer** includes a **recurrent connection** as part of its input. T
 - tprocessing, activation value of the hidden layer depends on the current input as well as the activation value of the hidden layer from the previous time step.
- The **hidden layer from the previous time step** provides a form of **memory**, or **context**, that encodes earlier processing, and decisions can be made at later points in time.
 - This approach does not impose a fixed-length limit on this prior context.
 - The context embodied in the previous hidden layer can include information extending back to the beginning of the sequence

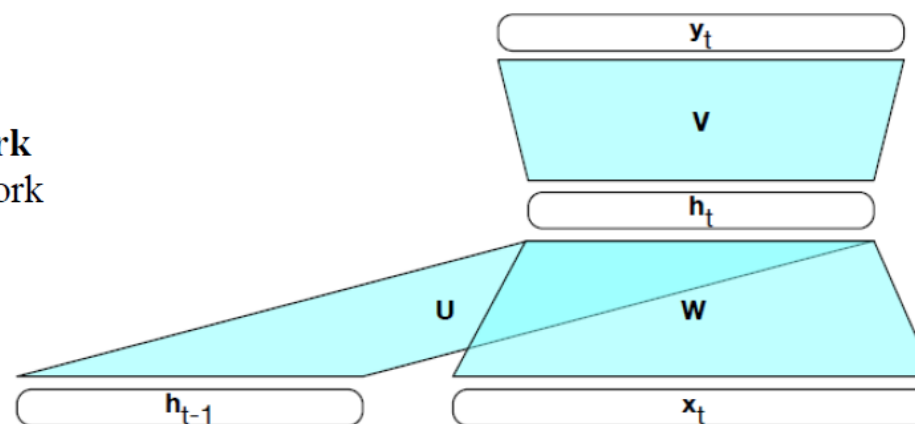


Recurrent Neural Networks (RNNs)

Inference in RNNs

- **Forward inference** (mapping a sequence of inputs to a sequence of outputs) in an RNN is nearly identical to what we've already seen with feedforward networks

Simple recurrent neural network
illustrated as a feedforward network



- Once we have the values for the hidden layer, we proceed with the usual computation to generate the output vector.

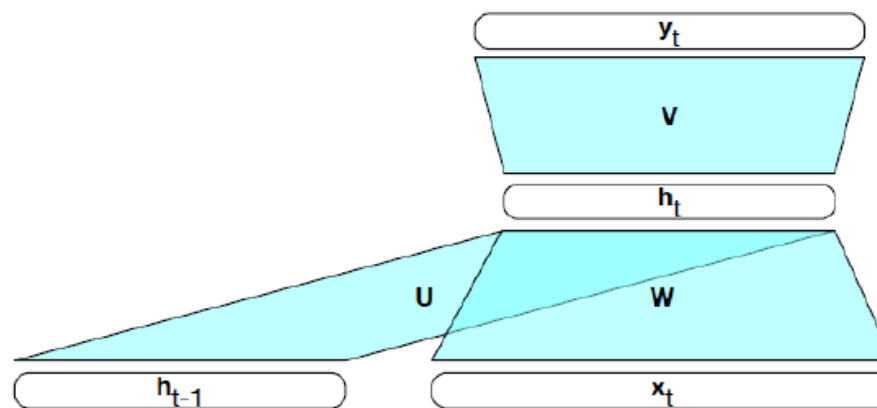
$$\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{x}_t)$$

$$\mathbf{y}_t = f(\mathbf{V}\mathbf{h}_t)$$

Recurrent Neural Networks (RNNs)

Inference in RNNs

```
function FORWARDRNN( $\mathbf{x}$ ,  $network$ )  
    returns output sequence  $\mathbf{y}$   
     $\mathbf{h}_0 \leftarrow 0$   
    for  $i \leftarrow 1$  to LENGTH( $\mathbf{x}$ ) do  
         $\mathbf{h}_i \leftarrow g(\mathbf{U}\mathbf{h}_{i-1} + \mathbf{W}\mathbf{x}_i)$   
         $\mathbf{y}_i \leftarrow f(\mathbf{V}\mathbf{h}_i)$   
    return  $\mathbf{y}$ 
```



- The matrices U , V and W are shared across time, while new values for $\mathbf{h}$ and $\mathbf{y}$ are calculated with each time step.

Recurrent Neural Networks (RNNs)

Inference in RNNs

```
function FORWARDRNN(x, network)
```

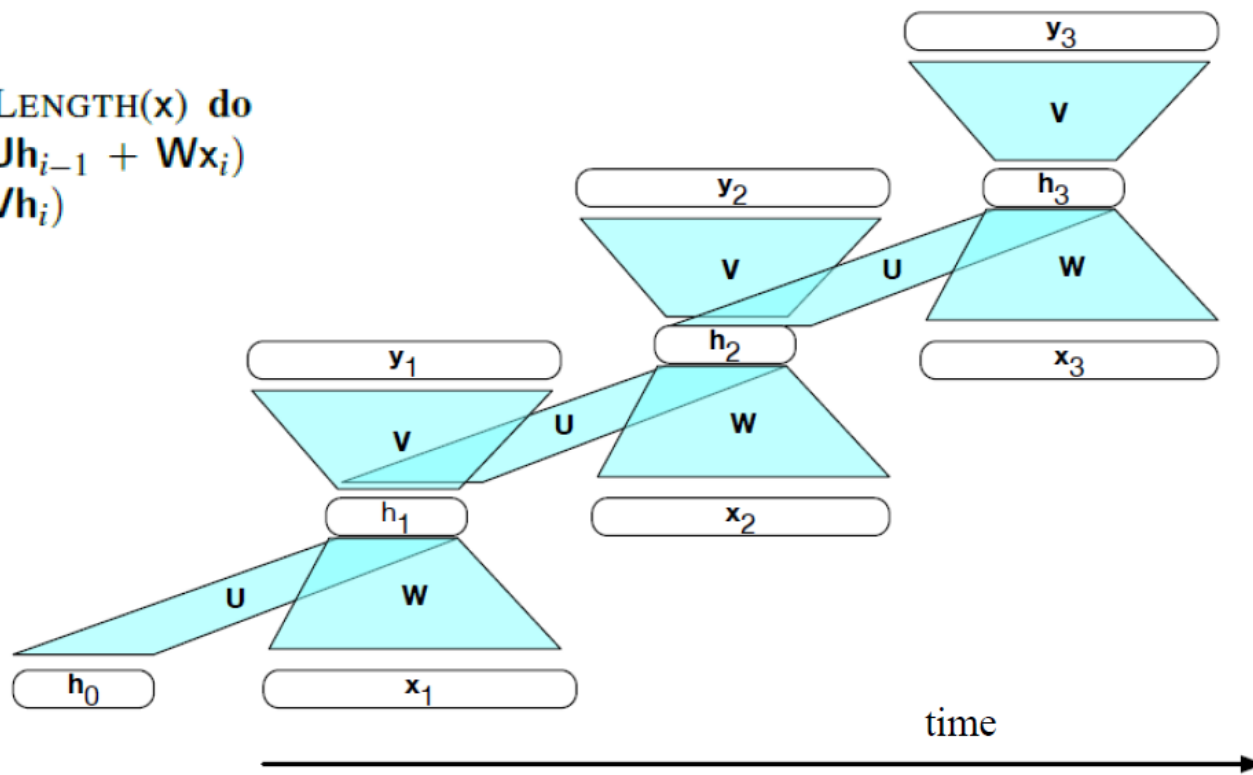
```
   $h_0 \leftarrow 0$ 
```

```
  for  $i \leftarrow 1$  to LENGTH(x) do
```

```
     $h_i \leftarrow g(Uh_{i-1} + Wx_i)$ 
```

```
     $y_i \leftarrow f(Vh_i)$ 
```

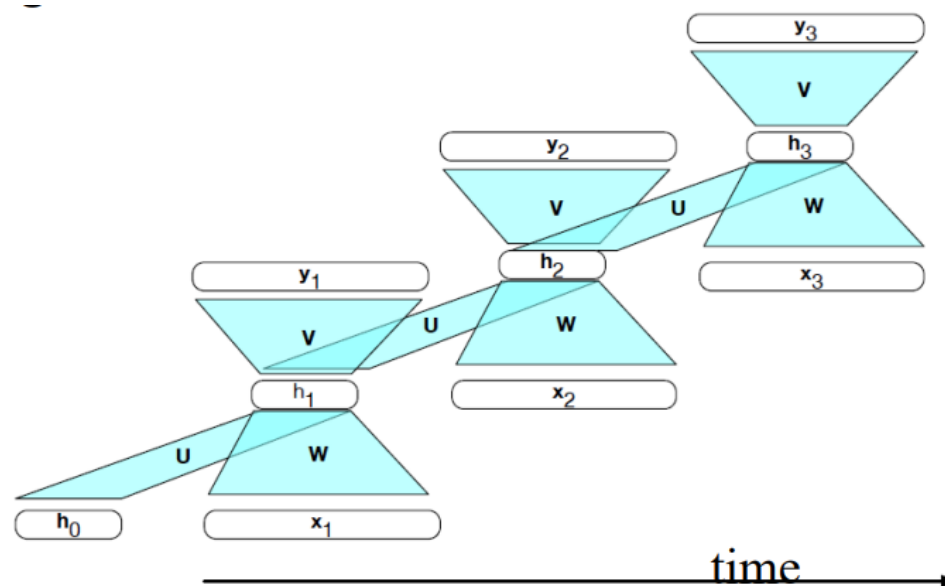
```
  return y
```



Recurrent Neural Networks (RNNs)

Training RNNs

- Fortunately, with modern computational frameworks and adequate computing resources, there is no need for a specialized approach to training RNNs.
- Enrolling a recurrent network into a feedforward computational graph eliminates any explicit recurrences, allowing the network weights to be trained directly.
- When presented with a specific input sequence, we can generate an unrolled feedforward network specific to that input, and use that graph to perform forward inference or training via ordinary backpropagation.

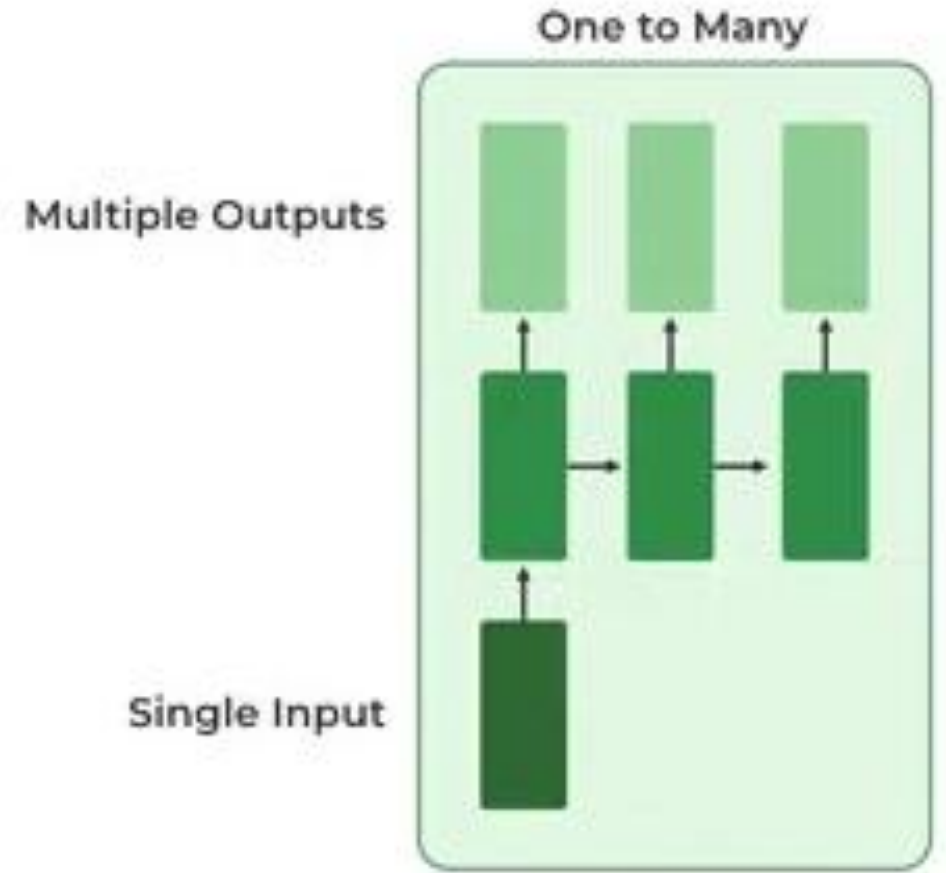


Types Of Recurrent Neural Networks

- One-to-One RNN
- One-to-Many RNN
- Many-to-One RNN
- Many-to-Many RNN

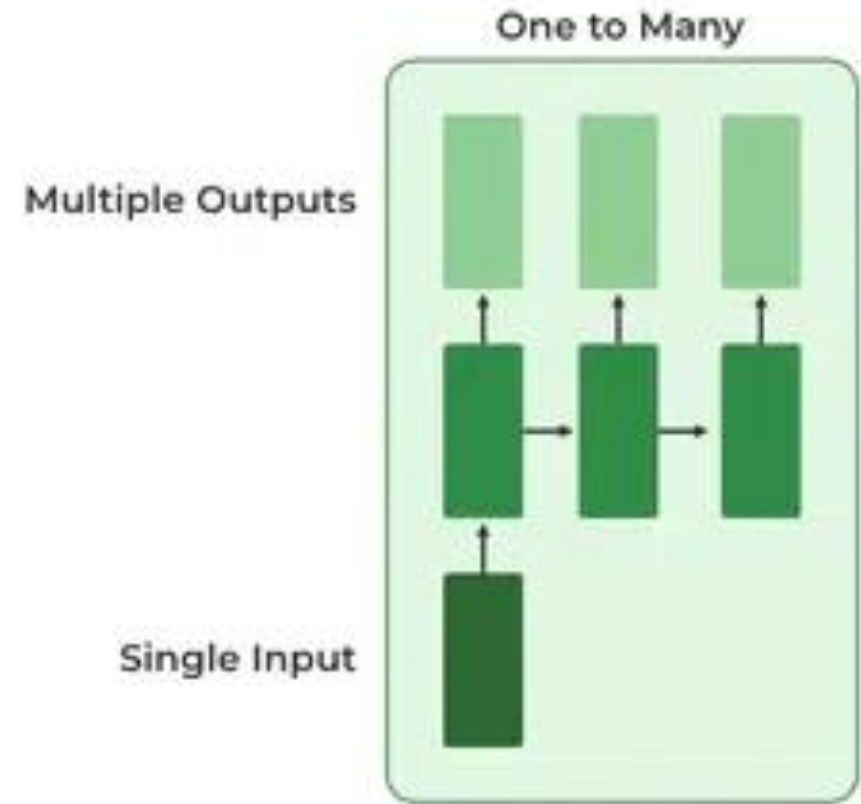
One-to-One RNN

- There is a single input and a single output. Commonly used for straightforward classification tasks where input data points do not depend on previous elements.



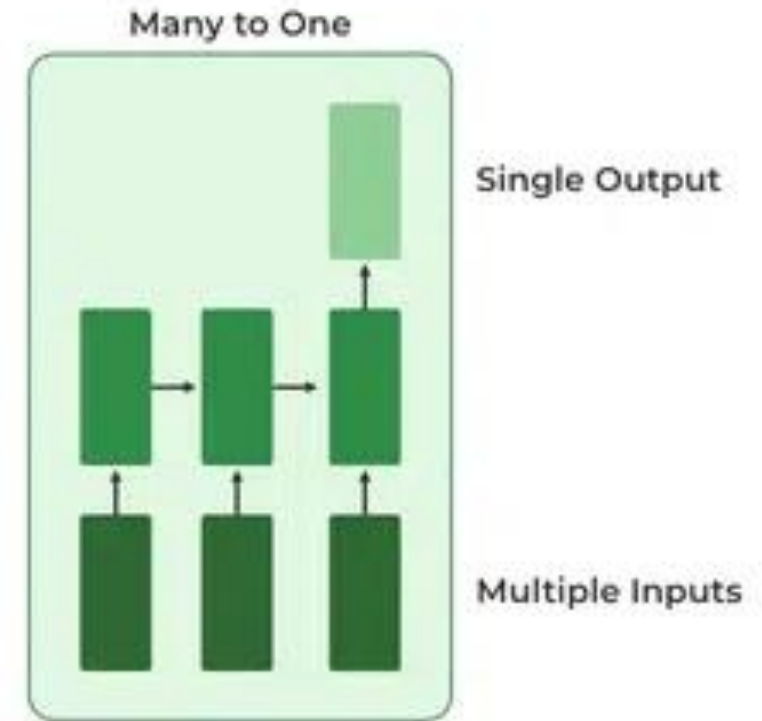
One-to-Many RNN

- In a One-to-Many RNN, the network processes a single input to produce multiple outputs over time. This setup is beneficial when a single input element should generate a sequence of predictions. For example, for image captioning task, a single image as input, the model predicts a sequence of words as a caption.



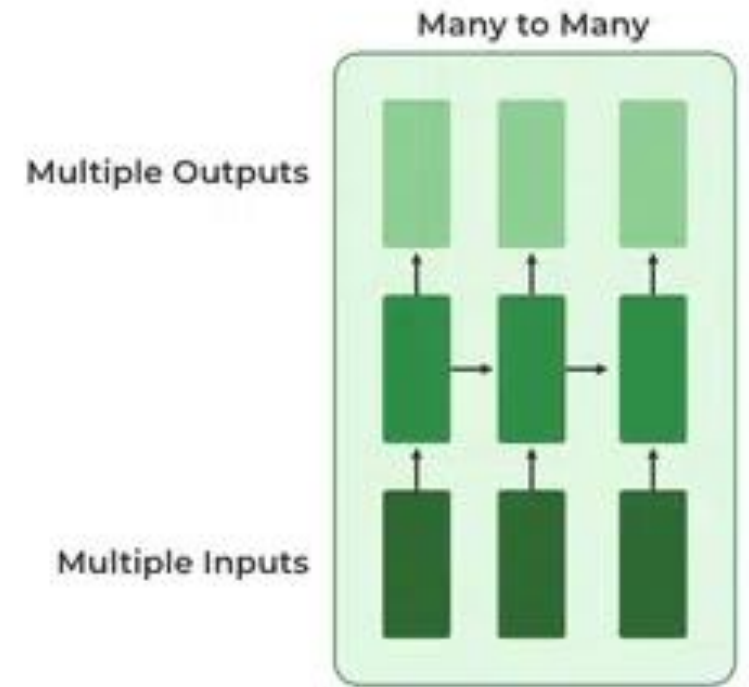
Many-to-One RNN

- The Many-to-One RNN receives a sequence of inputs and generates a single output. This type is useful when the overall context of the input sequence is needed to make one prediction. In sentiment analysis, the model receives a sequence of words (like a sentence) and produces a single output, which is the sentiment of the sentence (positive, negative, or neutral).



Many-to-Many RNN

- The Many-to-Many RNN type processes a sequence of inputs and generates a sequence of outputs. This configuration is ideal for tasks where the input and output sequences need to align over time, often in a one-to-one or many-to-many mapping. In language translation task, a sequence of words in one language is given as input, and a corresponding sequence in another language is generated as output.



Recurrent Neural Network Architectures

- Vanilla RNN
- Bidirectional RNNs
- Long Short-Term Memory Networks (LSTMs)
- Gated Recurrent Units (GRUs)

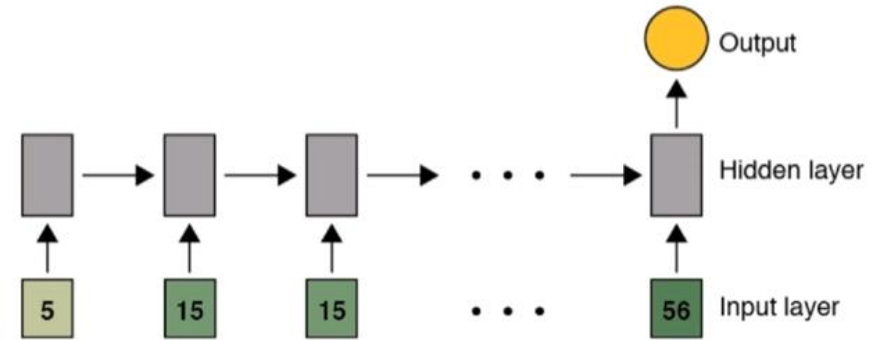
Vanilla RNN

- Basic RNN that processes sequences step-by-step, using hidden states to carry information. Effective for short sequences but struggles with long-term dependencies due to vanishing gradients.

Bidirectional RNNs

- Bidirectional RNNs process inputs in both forward and backward directions, capturing both past and future context for each time step. This architecture is ideal for tasks where the entire sequence is available, such as named entity recognition and question answering.

Code



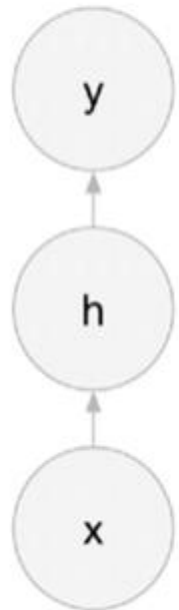
```
model = Model(inputs, outputs)
model.compile(loss='mse', optimizer='adam')
r = model.fit(
    Xtrain,
    Ytrain,
    epochs=50,
    validation_data=(Xtest, Ytest))
predictions = model.predict(Xtest)
```

Data is now $N \times T \times D$

```
# Things you should automatically know and have memorized
# N = number of samples
# T = sequence length
# D = number of input features
# M = number of hidden units
# K = number of output units
```

Code*

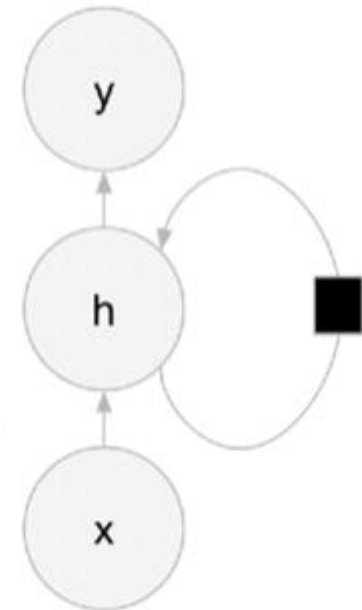
```
# Things you should automatically know and have memorized  
# N = number of samples  
# T = sequence length  
# D = number of input features  
# M = number of hidden units  
# K = number of output units
```



```
i = Input(shape=(D,))  
x = Dense(M, activation='relu')(i)  
x = Dense(K)(x)  
model = Model(i, x)
```

Default activation: tanh

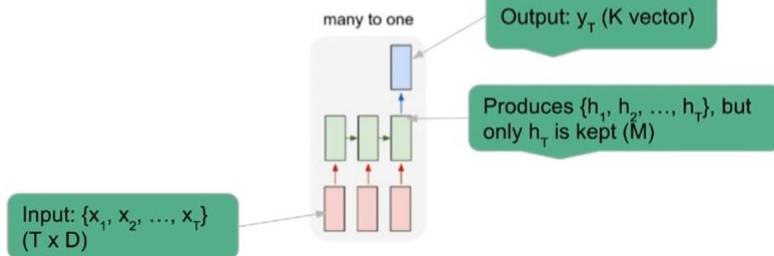
```
i = Input(shape=(T, D))  
x = SimpleRNN(M)(i)  
x = Dense(K)(x)  
model = Model(i, x)
```



Code

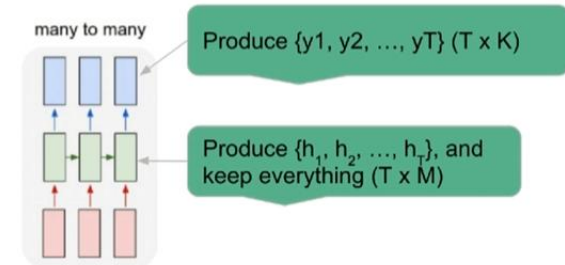
Many-to-One

```
i = Input(shape=(T, D))
x = SimpleRNN(M)(i)
x = Dense(K)(x)
model = Model(i, x)
```



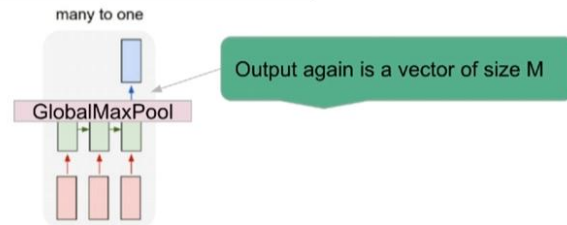
Many-to-Many

```
i = Input(shape=(T, D))
x = SimpleRNN(M, return_sequences=True)(i)
x = Dense(K)(x)
model = Model(i, x)
```



Many-to-One (again)

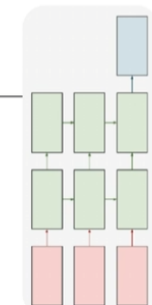
```
i = Input(shape=(T, D))
x = SimpleRNN(M, return_sequences=True)(i)
x = GlobalMaxPooling1D()(x)
x = Dense(K)(x)
model = Model(i, x)
```



Stacking Layers

```
i = Input(shape=(T, D))
x = SimpleRNN(32, return_sequences=True)(i)
x = SimpleRNN(32)(x) # default is return_sequences=False
x = Dense(K)(x)
model = Model(i, x)
```

many to many stacked



Outputs: $T \times 32$

Things you should automatically know and have memorized
N = number of samples
T = sequence length
D = number of input features
M = number of hidden units
K = number of output units

Code

Time Series vs. Categorical Sequence Inputs

Continuous Sequence

```
i = Input(shape=(T, D))
```

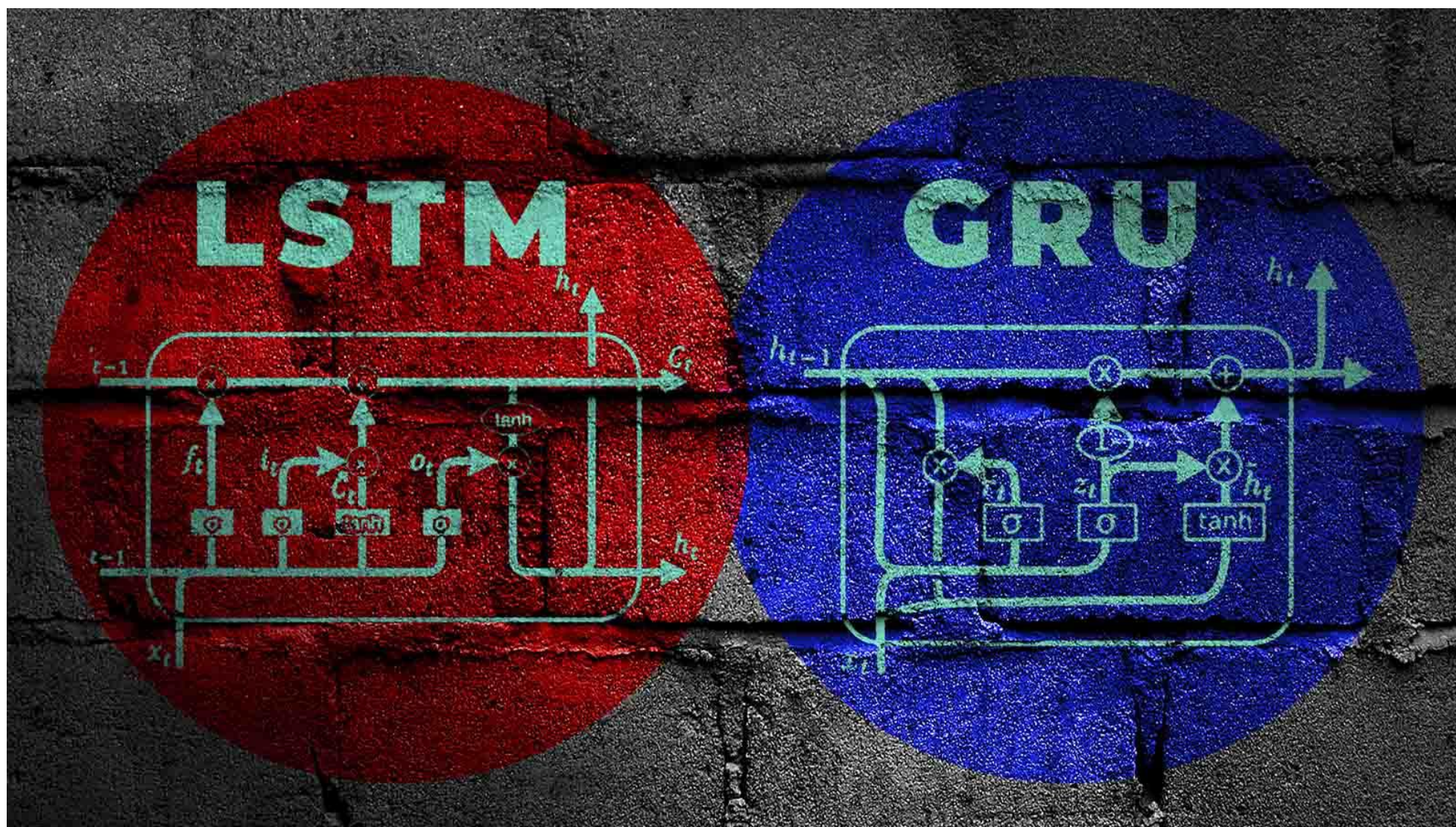
```
# at this point, input is T x D  
x = SimpleRNN(M) (i)  
x = Dense(K) (x)
```

Categorical Sequence

```
i = Input(shape=(T,))  
x = Embedding(V, D) (i)
```

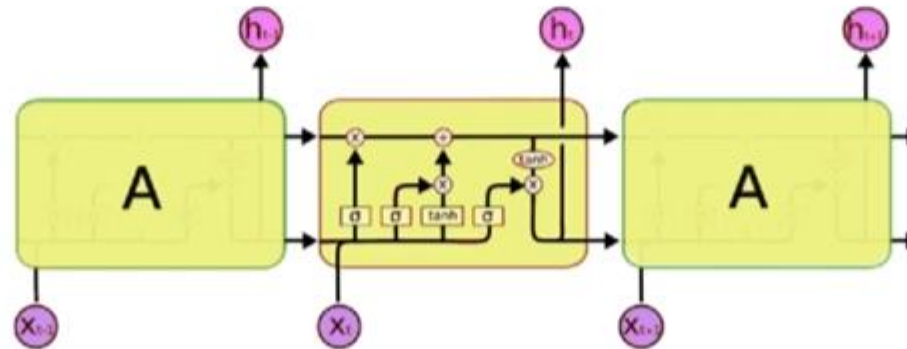
```
# at this point, input is T x D  
x = SimpleRNN(M) (x)  
x = Dense(K) (x)
```

It's easier to think of the input as a TxD "time series" - the rest of the RNN is the same in both cases!



Modern RNN ☺

- LSTM - Long Short-Term Memory
- GRU - Gated Recurrent Unit
- GRU is like a simplified version of the LSTM (less parameters and thus more efficient)
- Why do we need these at all?



Modern RNN ☺

- Consider again the vanishing gradient problem
- The output prediction is a huge composite function, depending on $x_1, x_2, \dots, x_T$
- W_{xh} appears several times (once at each time step)

$$\hat{y}(T) = \sigma(W_o^T \sigma(W_{xh}^T x_T + W_{hh}^T \sigma(W_{xh}^T x_{T-1} + W_{hh}^T \sigma(\dots(W_{xh}^T x_1 + W_{hh}^T h_0 \dots) \dots))))$$

- Don't think about what the actual derivatives *are*
- Just think about *which ones*
- We'll need the gradient of $W_{xh}^T x_t$ for all $t=1\dots T$
- The final gradient will be a function of all these individual gradients

$$\frac{\partial J}{\partial W_{xh}} = f\left(\frac{\partial(W_{xh}^T x_T)}{\partial W_{xh}}, \frac{\partial(W_{xh}^T x_{T-1})}{\partial W_{xh}}, \dots, \frac{\partial(W_{xh}^T x_1)}{\partial W_{xh}}\right)$$

Why modern RNNs?

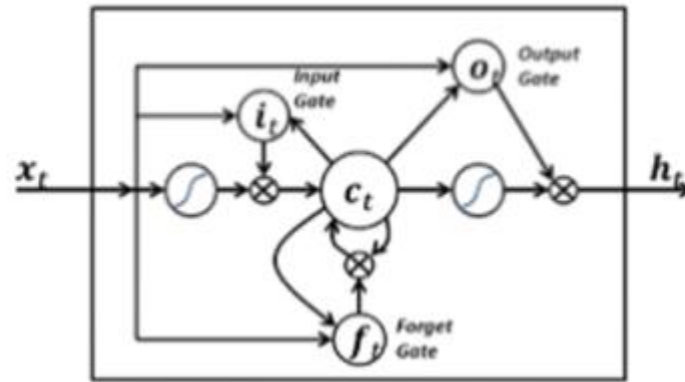
- Ask your simple RNN: "On what day was Albert Einstein born?"
- By the time the RNN has "read" the whole article, it forgot what was at the beginning!

- Consider how *deeply nested* each term is
- x_1 is the most deeply nested, x_2 next, etc.
- What do we remember about ANNs? ANN is a big composite function - composite functions turn into multiplications in the derivative (chain rule)
- The more deeply nested, the more multiplications
- RNNs are vulnerable to the vanishing gradient problem
- The farther back an input x_t is, the more its gradient vanishes
- The simple RNN can't learn from inputs too far back

$$\hat{y}(T) = \sigma(W_o^T \sigma(W_{xh}^T x_T + W_{hh}^T \sigma(W_{xh}^T x_{T-1} + W_{hh}^T \sigma(\dots(W_{xh}^T x_1 + W_{hh}^T h_0 \dots)\dots)))$$

Lesson - I

- Created in 1997 (even before the field was called “deep learning”)
- Lesson: research interesting ideas, don't simply chase what is popular
- You would have never discovered LSTMs!
- Over a decade old and part of some niche and relatively unknown field
- Go for “interesting”, not for “popular”
- GRU invented in 2014 - uses the same principles, but simpler



Long Short-Term Memory Networks (LSTMs)

- LSTMs are a type of RNN designed to overcome the problem of vanishing gradients, which can occur in vanilla RNNs. LSTMs introduce memory cells and gates (input, output, and forget gates) that control the flow of information, allowing the network to retain or forget information over long sequences. LSTMs are widely used in applications like language modeling, speech recognition, and time series forecasting.

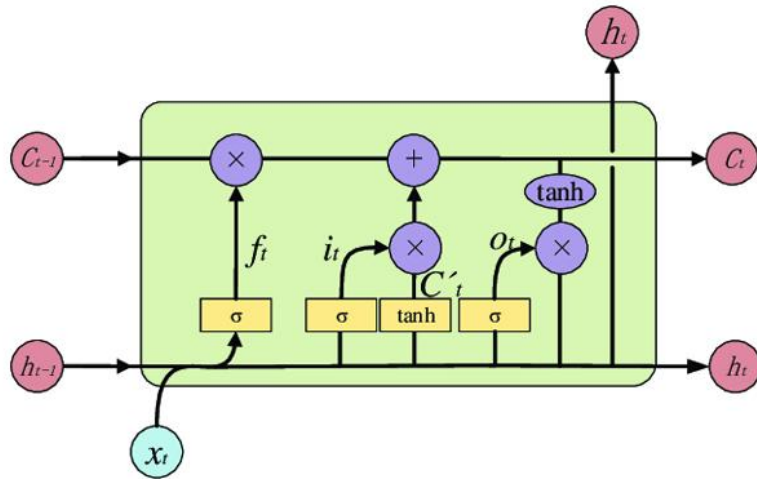
LSTM

- There are a total of three gates that LSTM uses as *Input Gate*, *Forget Gate*, and *Output Gate*.
- Input Gate :decides what information will be stored in long term memory.
- Forget Gate: decides which information from long term memory be kept or discarded.
- Output Gate: takes the current input, the previous short term memory and newly computed long term memory to produce new short term memory which will be passed on to the cell in the next time step.

LSTM



- LSTM returns 2 states:
 - Hidden state $h(t)$
 - Cell state $c(t)$ (usually ignored)
 - LSTM unit in Tensorflow outputs $h(T)$, but can also optionally output $c(T)$
 - Also means you need 2 initial states: c_0 and h_0



- Remember! Each of these is just a neuron

$$\begin{aligned}
 f_t &= \sigma(W_{xf}^T x_t + W_{hf}^T h_{t-1} + b_f) && \text{Forget gate} \\
 i_t &= \sigma(W_{xi}^T x_t + W_{hi}^T h_{t-1} + b_i) && \text{Input/update gate} \\
 o_t &= \sigma(W_{xo}^T x_t + W_{ho}^T h_{t-1} + b_o) && \text{Output gate} \\
 c_t &= f_t \odot c_{t-1} + i_t \odot f_c(W_{xc}^T x_t + W_{hc}^T h_{t-1} + b_c) \\
 h_t &= o_t \odot f_h(c_t) && \text{"Simple RNN"}
 \end{aligned}$$

LSTM

$f(t)$ = Neuron (binary classifier)

$i(t)$ = Neuron (binary classifier)

$o(t)$ = Neuron (binary classifier)

$c(t) = f(t) * c(t-1) + i(t) * \text{SimpleRNN}$

$h(t) = o(t) * \tanh(c(t))$

In Code

Simple RNN

```
i = Input(shape=(T,D))  
x = SimpleRNN(M)(i)  
x = Dense(K)(x)  
model = Model(i, x)
```

LSTM

```
i = Input(shape=(T,D))  
x = LSTM(M)(i)  
x = Dense(K)(x)  
model = Model(i, x)
```


Gated Recurrent Units (GRUs)

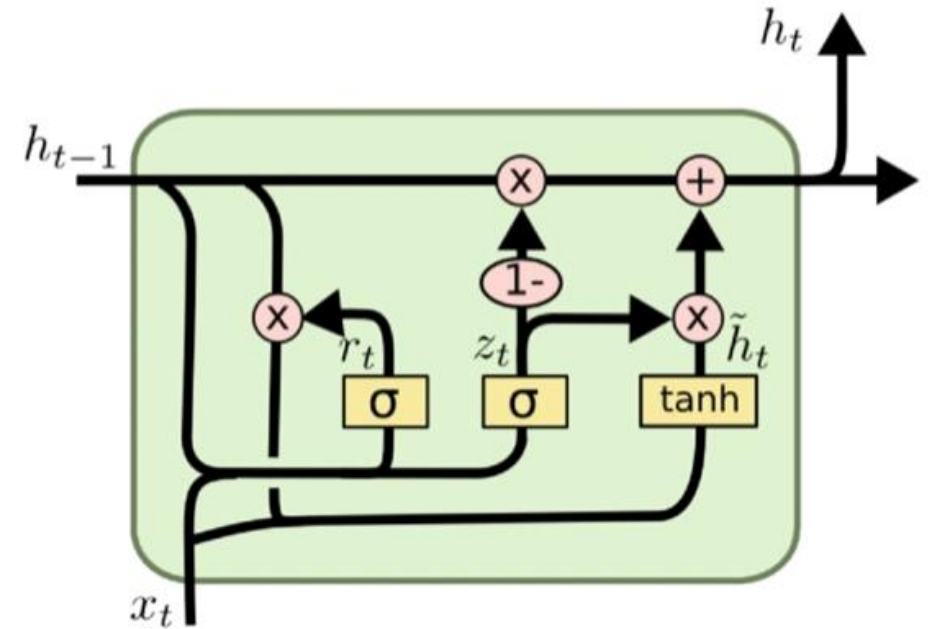
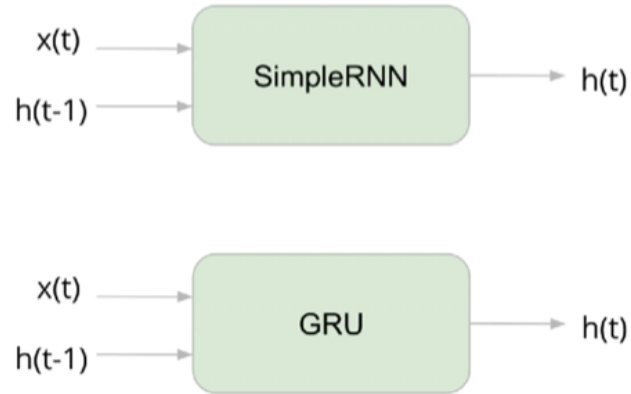
- GRUs are a simplified version of LSTMs, combining the forget and input gates into a single update gate. While GRUs are simpler and faster than LSTMs, they perform comparably in many tasks. Like LSTMs, GRUs are effective at learning long-term dependencies in sequential data.

GRU

- GRU incorporates the two gate operating mechanisms called **Update gate** and **Reset gate**.
- Update gate: responsible for determining the amount of previous information that needs to pass along the next state.
- Reset gate: decides how much of the past information is needed to neglect; in short, it decides whether the previous cell state is important or not.
- First, the reset gate comes into action it stores relevant information from the past time step into new memory content.
- Then it multiplies the input vector and hidden state with their weights.
- Next, it calculates element-wise multiplication between the reset gate and previously hidden state multiple.
- After summing up the above steps the non-linear activation function is applied and the next sequence is generated.

GRU

- First and foremost, it has the same “API” as the SimpleRNN



$$z_t = \sigma(W_{xz}^T x_t + W_{hz}^T h_{t-1} + b_z)$$

$$r_t = \sigma(W_{xr}^T x_t + W_{hr}^T h_{t-1} + b_r)$$

$$h_t = (1 - z_t) \odot h_{t-1} + z_t \odot \tanh(W_{xh}^T x_t + W_{hh}^T (r_t \odot h_{t-1}) + b_h)$$

$z(t)$ = update gate vector
 $r(t)$ = reset gate vector
 $h(t)$ = hidden state

Circle with a dot
 is element-wise
 multiplication

Simple RNN unit, for comparison

$$h_t = \tanh(W_{xh}^T x_t + W_{hh}^T h_{t-1} + b_h)$$

Code

```
i = Input(shape=(T, D))  
x = LSTM(M)(i)  
x = Dense(K)(x)  
model = Model(i, x)
```

```
i = Input(shape=(T, D))  
x = GRU(M)(i)  
x = Dense(K)(x)  
model = Model(i, x)
```

Backpropagation Through Time (BPTT)

- Backpropagation Through Time (BPTT) is an extension of the standard backpropagation algorithm, designed specifically for training Recurrent Neural Networks (RNNs). It accounts for the sequential nature of RNNs, where each timestep depends on previous timesteps.

The basic steps of BPTT in RNNs

- **Forward Pass:** The hidden states and outputs are computed at each time step.
- **Compute Loss:** The computed outputs are compared to the desired outputs (targets) to calculate the loss. The choice of loss function depends on the specific task and the type of output.
- **Backward Pass:** Starting from the last time step, gradients are calculated with respect to the parameters of the network. The gradients capture the influence of each parameter on the final loss. The gradients are calculated using the chain rule, similar to traditional backpropagation.
- **Gradient Updates:** The gradients are used to update the network's parameters, such as the weight matrices and biases, in the direction that minimizes the loss. This update step is typically performed using an optimization algorithm like gradient descent or its variants.

Vanishing and Exploding Gradients

- **Vanishing Gradients:** Occur in RNNs when gradients shrink exponentially during backpropagation, slowing or halting learning as earlier layers fail to update effectively.
- **Exploding Gradients:** Occur when gradients grow uncontrollably during backpropagation, causing numerical instability and erratic parameter updates.

Recurrent Neural Networks (RNNs)

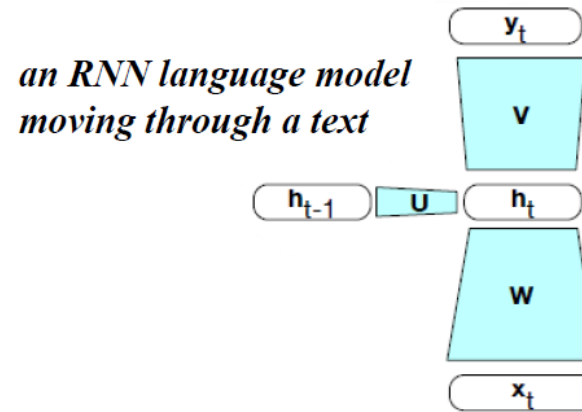
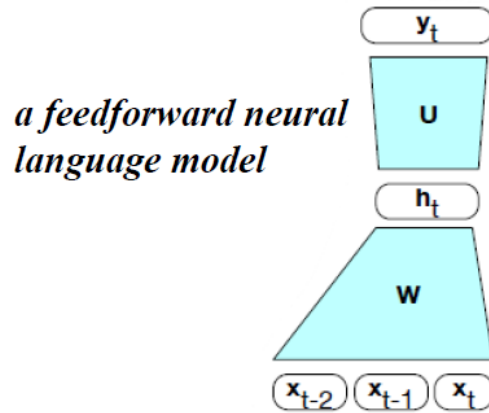
- *RNNs as Language Models*

RNNs as Language Models

- **Language models** predict the next word in a sequence given some preceding context.
- Language models can assign **a conditional probability to every possible next word**.
- They can also assign **probabilities to entire sequences** by combining these conditional probabilities with the chain rule:

$$P(w_{1:n}) = \prod_{i=1}^n P(w_i | w_{<i})$$

RNNs as Language Models



- The **n-gram language models** compute the probability of a word given counts of its occurrence with the $n-1$ prior words. The **context size is $n-1$** .
- For the **feedforward language models**, the **context size is the window size**.
- **RNNs thus don't have the limited context problem**, since the hidden state can in principle represent information about all of the preceding words.
 - RNN language models process the input sequence one word at a time, attempting to predict the next word from the current word and the previous hidden state.

RNNs as Language Models

Forward Inference

- **Forward inference** is the task, given an input, of running a forward pass on the network to produce a probability distribution over possible outputs (next words).
- The **input sequence** $X=[x_1, \dots, x_n]$ consists of a series of words each represented as a **one-hot vector** of size $|V| \times 1$,
- The **output prediction**, y , is a vector representing a probability distribution over the vocabulary.
- At each step, the model uses the word embedding matrix E to retrieve the embedding for the current word, and then combines it with the hidden layer from the previous step to compute a new hidden layer.
- This hidden layer is then used to generate an output layer which is passed through a **softmax** layer to generate a probability distribution over the entire vocabulary.

RNNs as Language Models

Forward Inference

- At time t :

$$\mathbf{e}_t = \mathbf{E}\mathbf{x}_t$$

$$\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{e}_t)$$

$$\mathbf{y}_t = \text{softmax}(\mathbf{V}\mathbf{h}_t)$$

- $\mathbf{E}$ is embeddings (size: $d_x|V|$), $\mathbf{x}_t$ is one-hot vector (size: $|V|\times 1$) for the word w_t in the input sequence, $\mathbf{e}_t$ is the embedding (size: $d_x\times 1$) of the word w_t .
- $\mathbf{h}_t$ is the hidden layer output at time t (size: $d_h\times 1$), $\mathbf{W}$ is the weight matrix for the input word (size: $d_h\times d_x$), $\mathbf{U}$ is the weight matrix for the context (size: $d_h\times d_h$).
- $\mathbf{V}$ is the weight matrix for the output layer (size: $|V|\times d_h$).
- $\mathbf{V}\mathbf{h}_t$ can be thought of as a set of scores over the vocabulary given the evidence provided in $\mathbf{h}_t$.
- $\mathbf{y}_t$: Passing these scores through the softmax normalizes the scores into a probability distribution over the vocabulary.

RNNs as Language Models

Forward Inference

- At time t :
 $\mathbf{e}_t = \mathbf{E}\mathbf{x}_t$
 $\mathbf{h}_t = g(\mathbf{U}\mathbf{h}_{t-1} + \mathbf{W}\mathbf{e}_t)$
 $\mathbf{y}_t = \text{softmax}(\mathbf{V}\mathbf{h}_t)$
- The probability that a particular word k in the vocabulary is the next word at time t is represented by $\mathbf{y}_t[k]$, the k^{th} component of $\mathbf{y}_t$:

$$P(w_{t+1} = k | w_1, \dots, w_t) = \mathbf{y}_t[k]$$

- The probability of an entire sequence is just the product of the probabilities of each item in the sequence, where $\mathbf{y}_i[w_i]$ is the probability of the true word w_i at time step i .

$$P(w_{1:n}) = \prod_{i=1}^n P(w_i | w_{1:i-1}) = \prod_{i=1}^n \mathbf{y}_i[w_i]$$

RNNs as Language Models

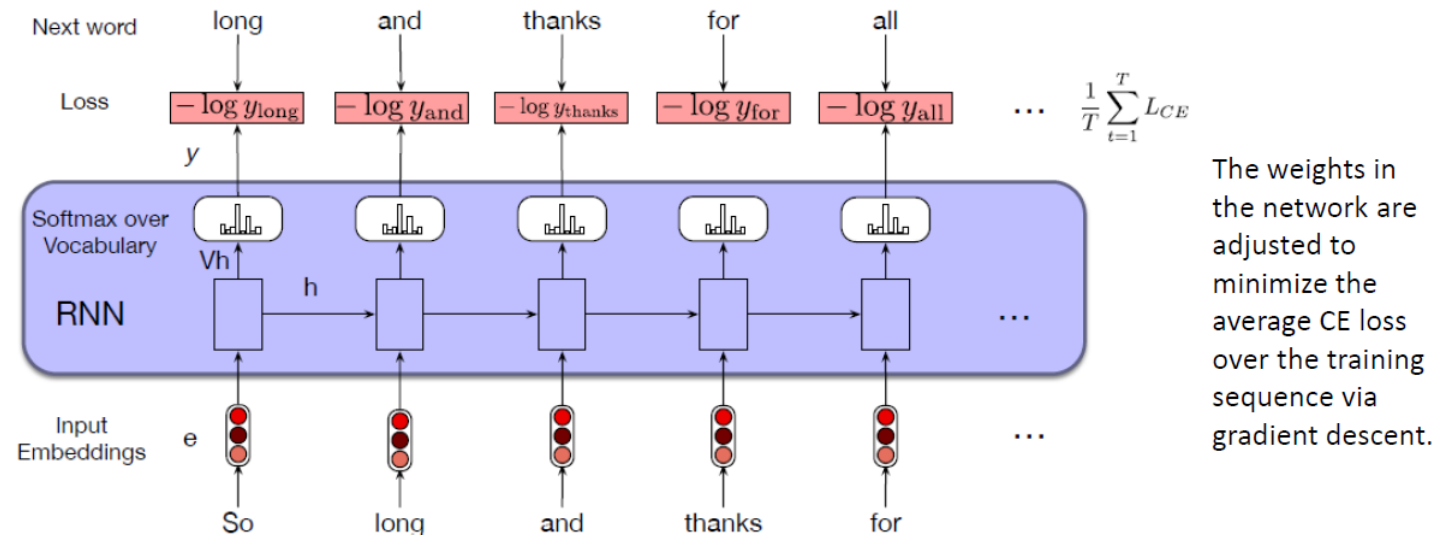
Training an RNN language model

- **self-supervision:** To train an RNN as a language model, we take a corpus of text as training material and at each time step t ask the model to predict the next word.
- We train the model to minimize the error in predicting the true next word in the training sequence, using cross-entropy as the loss function.
 - Recall that the cross-entropy loss measures the difference between a predicted probability distribution and the correct distribution.
 - The correct distribution y_t comes from knowing the next word.
 - At time t the CE loss is the negative log probability the model assigns to the next word in the training sequence.

$$L_{CE}(\hat{\mathbf{y}}_t, \mathbf{y}_t) = -\log \hat{\mathbf{y}}_t[w_{t+1}]$$

RNNs as Language Models

Training an RNN language model



- At each word position t , the model takes the correct sequence of tokens $w_{1:t}$, and computes a probability distribution over possible next words to compute model's loss for next token w_{t+1} .
 - Then we ignore what the model predicted for the next word and instead use the correct sequence of tokens $w_{1:t+1}$ to estimate the probability of token w_{t+2} .
 - The idea that the model use the correct history sequence to predict the next word (rather than feeding the model its best case from the previous time step) is **called teacher forcing**.

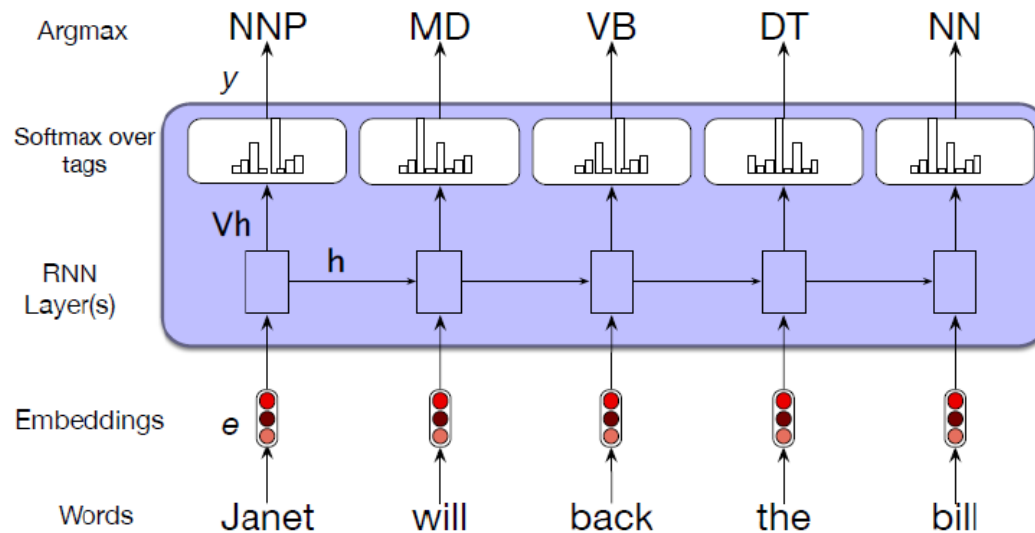
Recurrent Neural Networks (RNNs)

- *RNNs for other NLP tasks*

RNNs for Other NLP Tasks

Sequence Labeling

- In **sequence labeling**, the network's task is to assign a label chosen from a small fixed set of labels to each element of a sequence, like the **part-of-speech tagging**.
- **Part-of-speech tagging as sequence labeling with a simple RNN.**
 - Pre-trained word embeddings serve as inputs and a softmax layer provides a probability distribution over the part-of-speech tags as output at each time step.



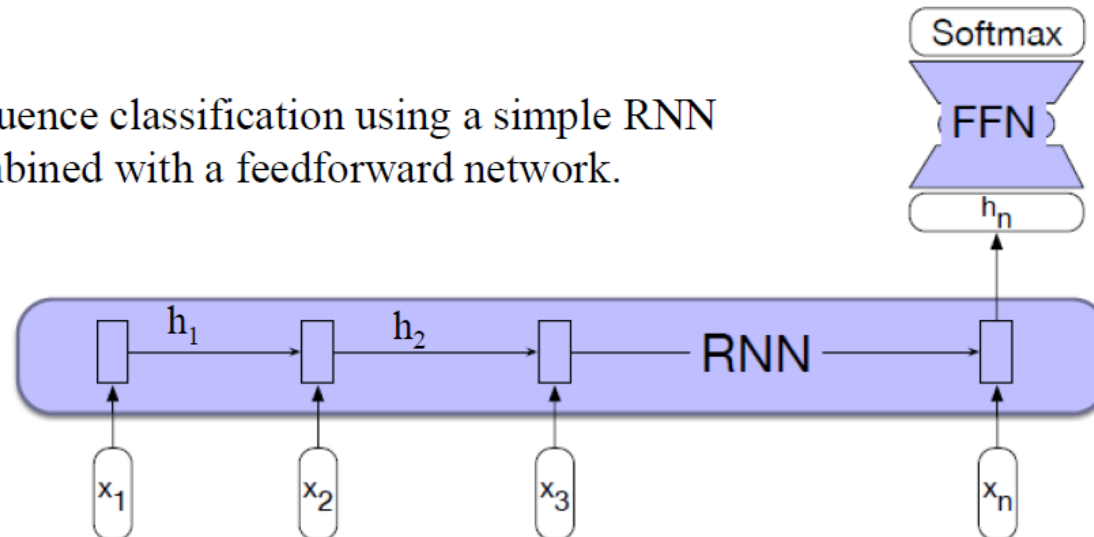
This RNN represents an **unrolled simple recurrent network** consisting of an input layer, hidden layer, and output layer at each time step, as well as the shared U , V and W weight matrices that comprise the network.

RNNs for Other NLP Tasks

RNNs for Sequence Classification – Text Classification

- RNNs can **classify entire sequences** rather than the tokens within them.
- The text to be classified is passed through the RNN a word at a time generating a new hidden layer at each time step.
- The hidden layer for the last token of the text, h_n , to constitute a compressed representation of the entire sequence.
- h_n is given to a feedforward network that chooses a class via a softmax function.

Sequence classification using a simple RNN combined with a feedforward network.

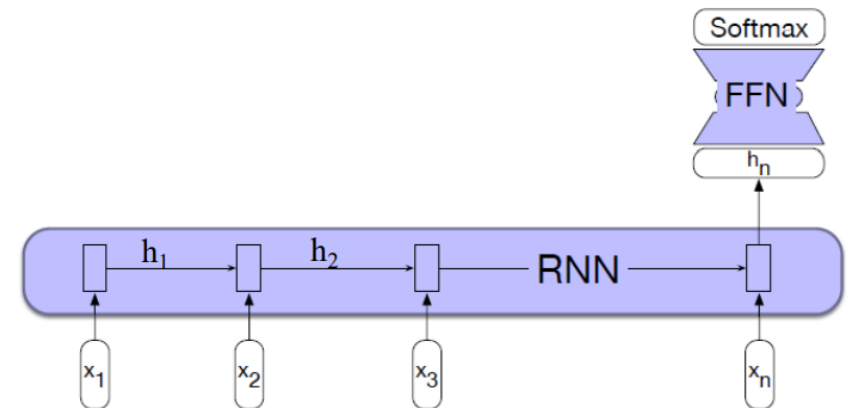


RNNs for Other NLP Tasks

RNNs for Sequence Classification – Text Classification

- No need intermediate outputs for the words in the sequence preceding the last element.
 - Therefore, there are no loss terms associated with those elements.
 - Instead, the loss function used to train the weights in the network is based entirely on the final text classification task.
 - The output from the softmax output from the feedforward classifier together with a crossentropy loss drives the training.
 - The error signal from the classification is backpropagated all the way through the weights in the feedforward classifier through, and then through to the three sets of weights in the RNN
- Instead of using just last token h_n to represent the whole pooling sequence, we can use a **pooling function** of all hidden states h_i .
- A representation that pools all the n hidden states by taking their element-wise mean:

$$h_{mean} = \frac{1}{n} \sum_{i=1}^n h_i$$



RNNs for Other NLP Tasks

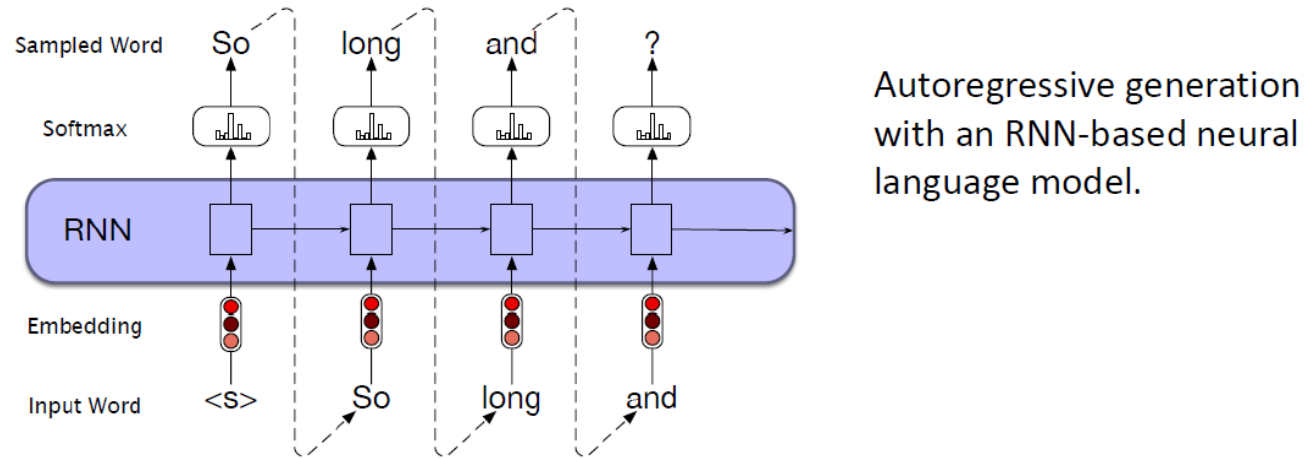
Generation with RNN-Based Language Models

- **RNN-based language models can also be used to generate text.**
 - Text generation is of enormous practical importance any task where a system needs to produce text such as question answering, machine translation, text summarization.
 - Text generation constitute a new area of AI that is often called **generative AI**.
- Using a language model to incrementally generate words by repeatedly sampling the next word conditioned on our previous choices is called **autoregressive generation**.

- Sample a word in the output from the softmax distribution that results from using the beginning of sentence marker, <s>, as the first input.
- Use the word embedding for that first word as the input to the network at the next time step, and then sample the next word in the same fashion.
- Continue generating until the end of sentence marker, </s>, is sampled or a fixed length limit is reached.

RNNs for Other NLP Tasks

Generation with RNN-Based Language Models



- Autoregressive generation architecture is applicable to many NLP applications such as machine translation, summarization, and question answering.
 - The key is to prime the generation component with an appropriate context.
 - Instead of simply using <s> to get things started we can provide a richer task-appropriate context; for translation the context is the sentence in the source language; for summarization it's the long text we want to summarize.

Recurrent Neural Networks (RNNs)

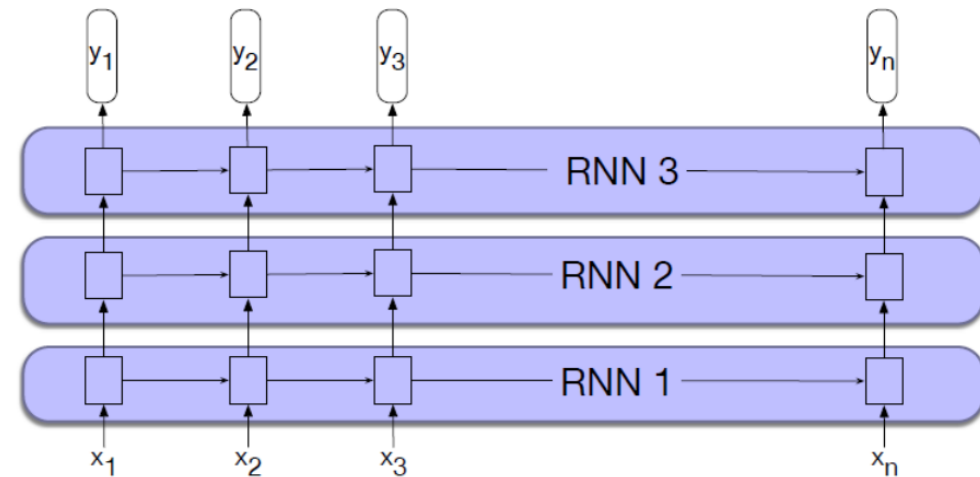
- *Stacked and Bidirectional RNN architectures*

Stacked RNNs

- Recurrent networks are quite flexible.
 - The entire sequence of outputs from one RNN as an input sequence to another one.
- **Stacked RNNs** consist of multiple networks where the output of one layer serves as the input to a subsequent layer.

Stacked recurrent networks.

- The output of a lower level serves as the input to higher levels with the output of the last network serving as the final output.
- Stacked RNNs generally outperform single-layer networks.
- As the number of stacks is increased the training costs rise quickly.

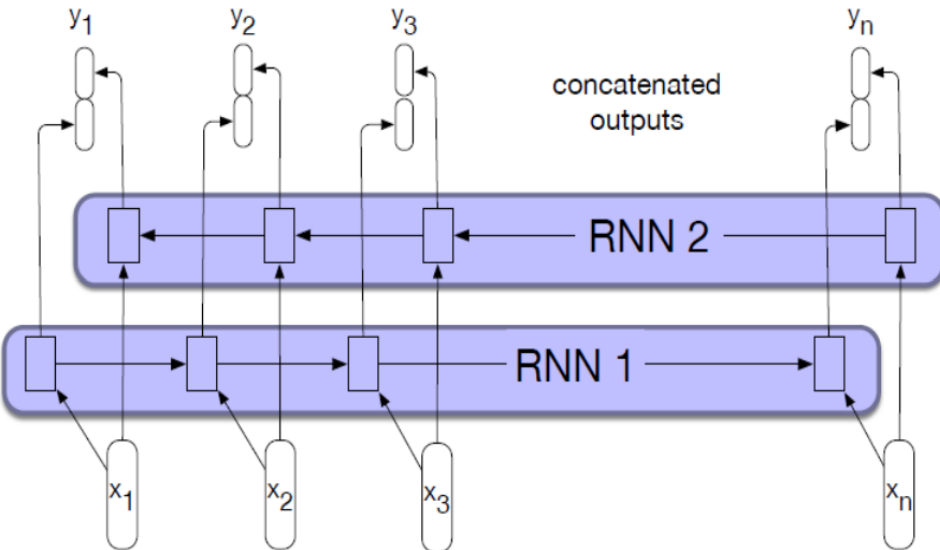


Bidirectional RNNs

- The RNN uses information from the left context to make its predictions at time t .
- But many NLP applications may require right context in addition to left context.
- A **bidirectional RNN** combines two independent RNNs, one where the input is processed from the start to the end, and the other from the end to the start.
 - We then concatenate the two representations computed by the networks into a single vector that captures both the left and right contexts of an input at each point in time.

A bidirectional RNN

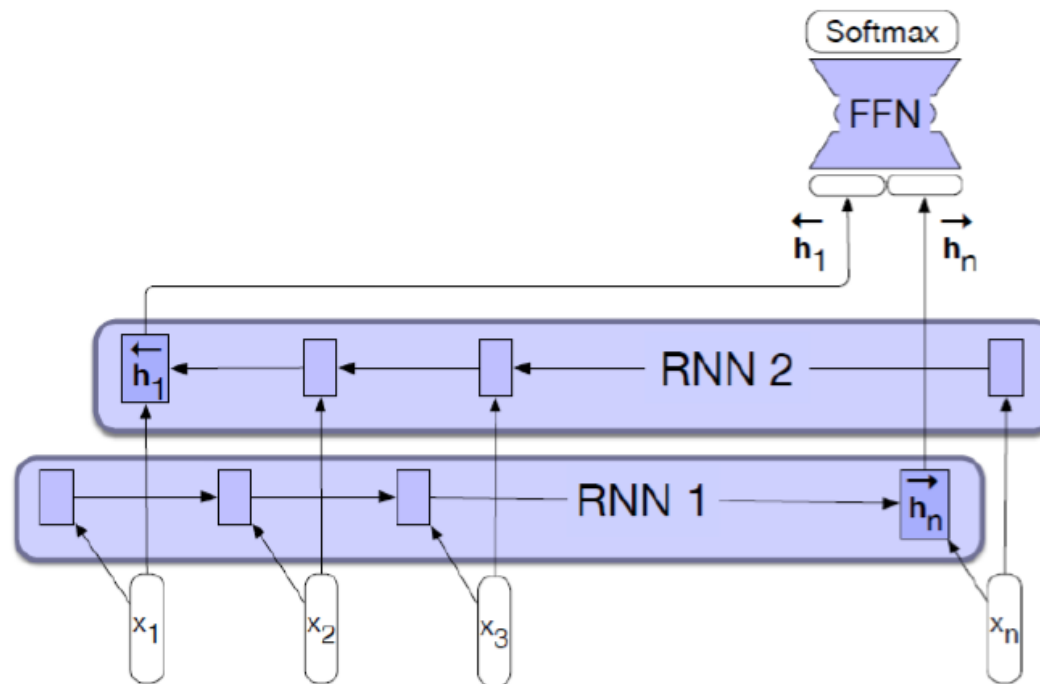
- Separate models are trained in the forward and backward directions, with the output of each model at each time point concatenated to represent the bidirectional state at that time point.



Bidirectional RNNs

A bidirectional RNN for sequence classification:

- The final hidden units from the forward and backward passes are combined to represent the entire sequence.
- This combined representation serves as input to the subsequent classifier.



LSTM (long short-term memory) Network

LSTM

shortcomings of RNNs – distant information

- **Distant information** is critical to many language applications.
- To train RNNs is difficult to make use of information distant from the current point of processing.
 - Although RNNs access to the entire preceding sequence, the information encoded in hidden states tends to be fairly local, more relevant to the most recent parts of the input.

Example: The flights the airline *was* canceling *were* full.

- Assigning a high probability to *was* following airline is straightforward since airline provides a strong **local context** for the singular agreement.
- However, assigning an appropriate probability to *were* is quite difficult, not only because the plural flights is quite **distant**, but also because the singular noun airline is closer in the intervening context.
- Ideally, **a network should be able to retain the distant information about plural flights until it is needed**, while still processing the intermediate parts of the sequence correctly.

LSTM

shortcomings of RNNs – distant information

Reasons for the inability of RNNs to carry forward critical information

1. The weights in the hidden layer are being asked to perform two tasks simultaneously:
 - provide information useful for the current decision, and
 - updating and carrying forward information required for future decisions.
2. Training RNNs needs to backpropagate the error signal back through time.
 - The hidden layer at time t contributes to the loss at the next time step since it takes part in that calculation.
 - During the **backward pass of training**, the hidden layers are subject to repeated multiplications, as determined by the length of the sequence.
 - A frequent result of this process is that the gradients are eventually driven to zero, a situation called the **vanishing gradients problem**.
- More complex network architectures are designed to explicitly manage the task of maintaining relevant context over time,
 - by enabling the network to learn to **forget information that is no longer needed** and
 - to **remember information required for decisions still to come**.

→ LSTM

LSTM

- The **long short-term memory (LSTM)** networks which are extensions of RNNs are designed **to solve context management problem**.
- LSTMs divide the context management problem into two subproblems:
 - **removing information no longer needed from the context, and**
 - **adding information likely to be needed for later decision making.**
- LSTMs accomplish the context management problem
 - Adding an **explicit context layer** to the architecture (in addition to hidden layer), and
 - Use of specialized neural units that make use of **gates to control the flow of information** into and out of the units that comprise the network layers.
 - These gates are implemented through the use of additional weights that operate sequentially on the input, and previous hidden layer, and previous context layers.

LSTM

gates

The **gates in an LSTM** share a common design pattern:

- Each consists of a **feedforward layer**, followed by a **sigmoid activation function**, followed by a **pointwise multiplication with the layer being gated**.
- The sigmoid activation function is preferred because it pushes its outputs to 0 or 1.
- A pointwise multiplication of the sigmoid function output and the layer being gated has an effect similar to that of a binary mask of the layer.
- Values in the layer being gated that align with values near 1 in the mask are passed through nearly unchanged; values corresponding to lower values are essentially erased.

LSTM

Gates – forget gate

- The purpose of **forget gate** is to delete information from the context that is no longer needed.
 - The forget gate computes a weighted sum of the previous state's hidden layer and the current input and passes that through a sigmoid.
 - This mask is then multiplied element-wise ($\odot$) by the context vector to remove the information from context that is no longer required.

$$\mathbf{f}_t = \sigma(\mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{W}_f \mathbf{x}_t)$$

$$\mathbf{k}_t = \mathbf{c}_{t-1} \odot \mathbf{f}_t$$

- $\mathbf{U}_f$ and $\mathbf{W}_f$: weights for the forget gate
- $\mathbf{c}_{t-1}$: the previous context
- $\mathbf{f}_t$: a binary mask for forget gate

LSTM

Gates – add gate

- The next task is to compute the actual information we need to extract from the previous hidden state and current inputs.

$$\mathbf{g}_t = \tanh(\mathbf{U}_g \mathbf{h}_{t-1} + \mathbf{W}_g \mathbf{x}_t)$$

- $\mathbf{U}_g$ and $\mathbf{W}_g$: weights for possible context items
- $\mathbf{g}_t$: possible context items to be added in the current state

- We generate the mask for the **add gate** to select the information to add to the current context. We add this to the modified context vector to get our new context vector.

$$\mathbf{i}_t = \sigma(\mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{W}_i \mathbf{x}_t)$$

$$\mathbf{j}_t = \mathbf{g}_t \odot \mathbf{i}_t$$

$$\mathbf{c}_t = \mathbf{j}_t + \mathbf{k}_t$$

- $\mathbf{U}_i$ and $\mathbf{W}_i$: weights for the add gate
- $\mathbf{i}_t$: a binary mask for add gate
- $\mathbf{j}_t$: the new context items to be added
- $\mathbf{c}_t$: the new context

LSTM

Gates – output gate

- The **output gate** is used to decide what information is required for the current hidden state.

$$\mathbf{o}_t = \sigma(\mathbf{U}_o \mathbf{h}_{t-1} + \mathbf{W}_o \mathbf{x}_t)$$

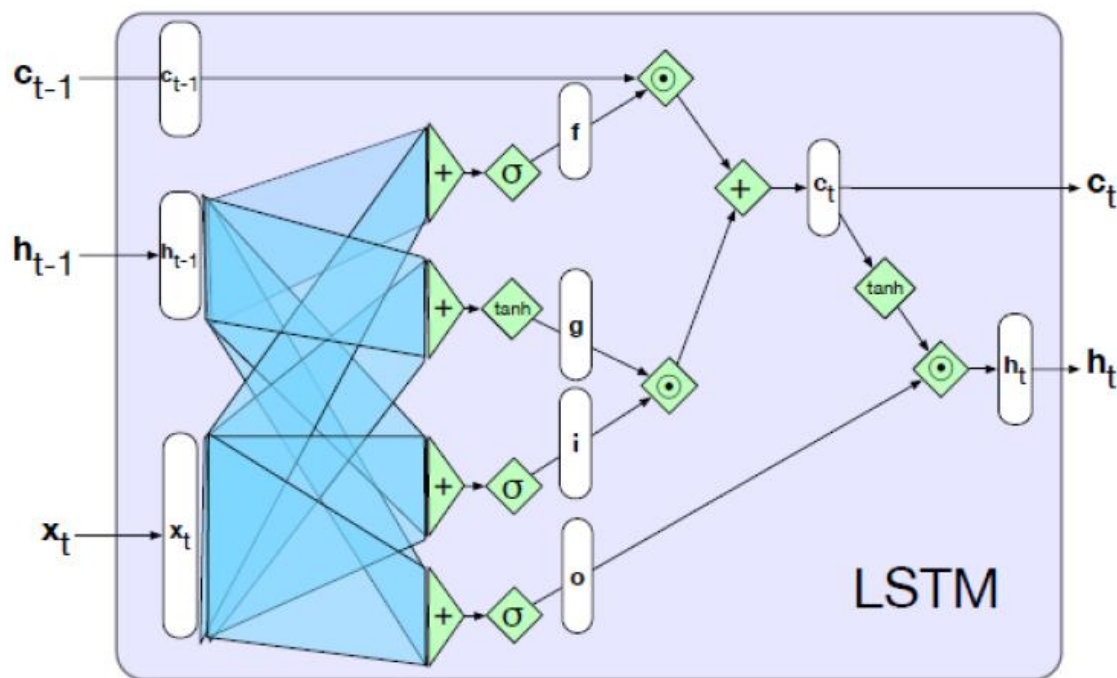
$$\mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t)$$

- $\mathbf{U}_o$ and $\mathbf{W}_o$: weights for output gate
- $\mathbf{o}_t$: binary mask for output gate
- $\mathbf{h}_t$: hidden layer at time t

LSTM

A single LSTM unit

- A single LSTM unit displayed as a computation graph:
 - The inputs to each unit consists of **current input x_t** , **previous hidden state h_{t-1}** , and **previous context c_{t-1}** .
 - The outputs are a **new hidden state h_t** and an **updated context c_t** .



forget gate

$$f_t = \sigma(\mathbf{U}_f \mathbf{h}_{t-1} + \mathbf{W}_f \mathbf{x}_t)$$

$$k_t = c_{t-1} \odot f_t$$

add gate

$$g_t = \tanh(\mathbf{U}_g \mathbf{h}_{t-1} + \mathbf{W}_g \mathbf{x}_t)$$

$$i_t = \sigma(\mathbf{U}_i \mathbf{h}_{t-1} + \mathbf{W}_i \mathbf{x}_t)$$

$$j_t = g_t \odot i_t$$

$$c_t = j_t + k_t$$

output gate

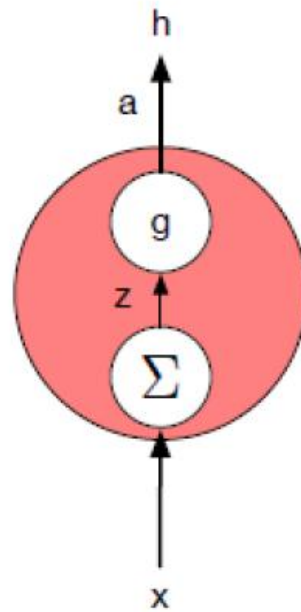
$$o_t = \sigma(\mathbf{U}_o \mathbf{h}_{t-1} + \mathbf{W}_o \mathbf{x}_t)$$

$$h_t = o_t \odot \tanh(c_t)$$

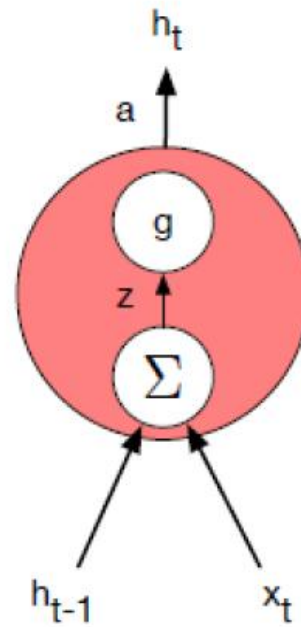
LSTM

LSTM units

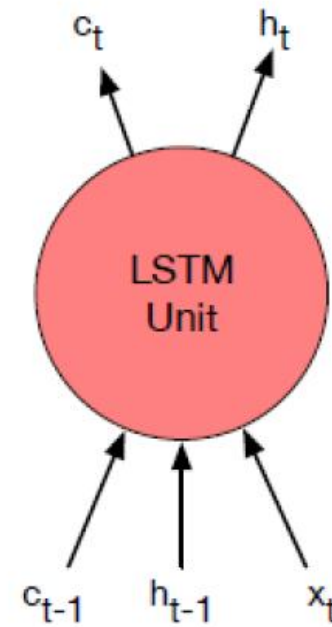
- The **neural units used in LSTMs** are obviously much more complex than those used in basic feedforward networks, simple RNNs.



Feedforward Unit



RNN Unit

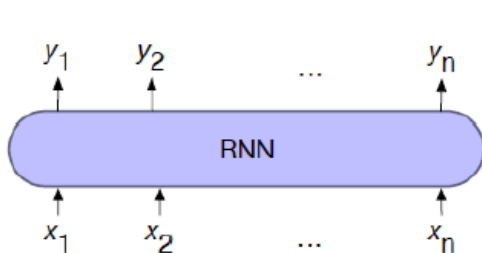


LSTM Unit

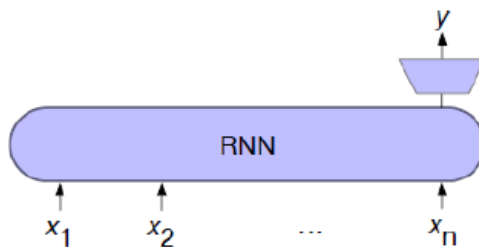
Encoder-Decoder Model with RNNs

Encoder-Decoder Model with RNNs

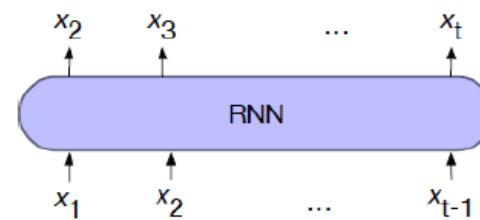
Four architectures for NLP tasks



In **sequence labeling** (POS tagging) we map each input token x_i to an output token y_i .



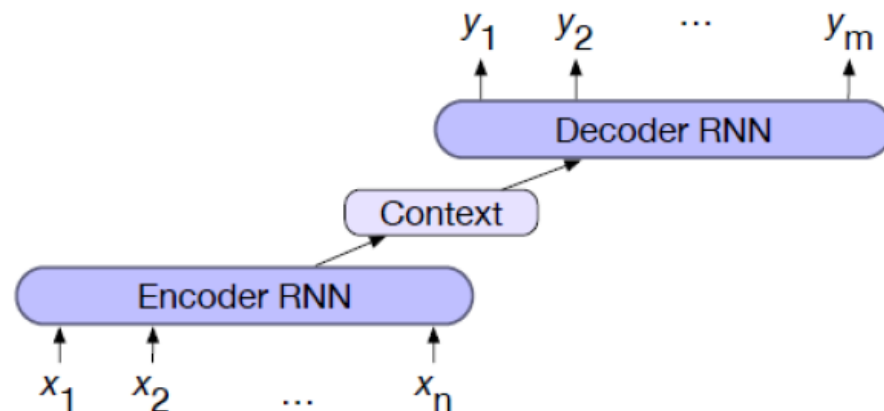
In **sequence classification** we map the entire input sequence to a single class.



In **language modeling** we output next token conditioned on previous tokens.

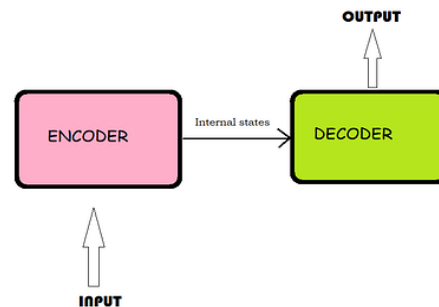
In **encoder-decoder** model we have two separate RNN models,

- First one maps input sequence to intermediate representation called as context, and
- Second one maps from context to output sequence.



Encoder-Decoder Model with RNNs

- The **encoder-decoder model** takes an input sequence and translates it to an output sequence that is of a different length than the input.
 - **Encoder-decoder models** are used especially for tasks like machine translation, where the input sequence and output sequence can have different lengths and the mapping between a token in the input and a token in the output can be very indirect.
- The key idea underlying these networks is the use of an encoder network that takes an input sequence and creates a contextualized representation of it, often called the **context**.
 - This context representation is then passed to a decoder which generates a task specific output sequence.

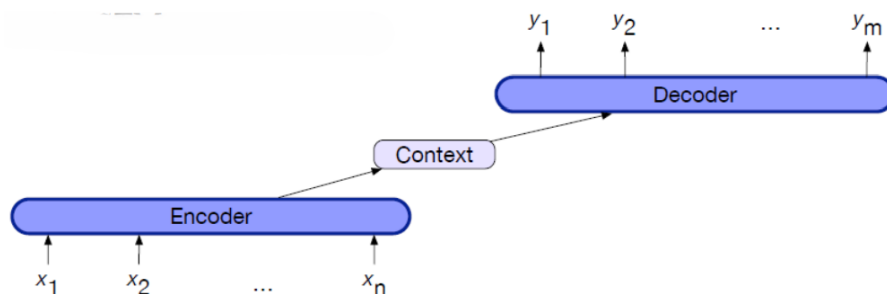


Encoder-Decoder Model with RNNs

The encoder-decoder architecture

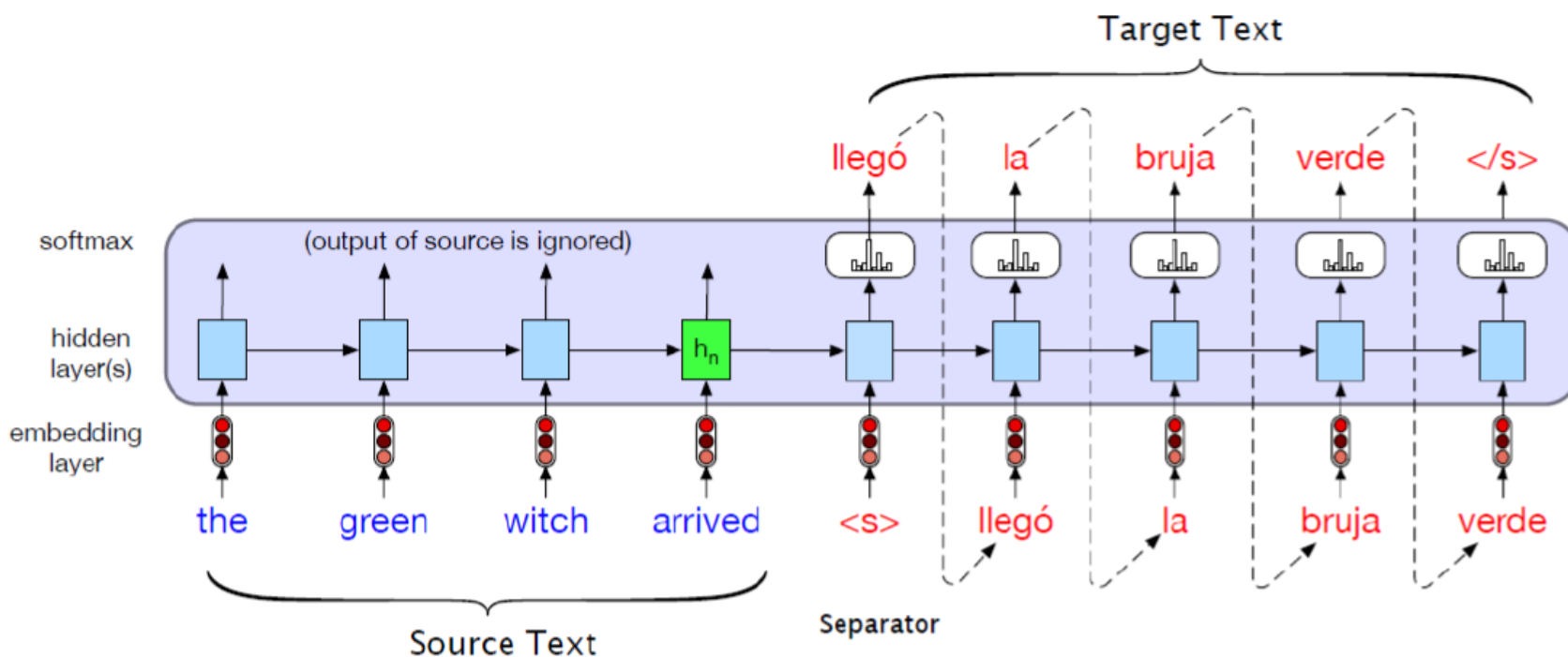
Encoder-decoder networks consist of **three conceptual components**:

1. An **encoder** that accepts an input sequence, $x_{1:n}$, and generates a corresponding sequence of contextualized representations, $h_{1:n}$.
 - LSTMs, convolutional networks, and transformers can all be employed as encoders.
2. A **context vector** c , which is a function of $h_{1:n}$, and conveys the essence of the input to the decoder.
3. A **decoder**, which accepts c as input and generates an arbitrary length sequence of hidden states $h_{1:m}$, from which a corresponding sequence of output states $y_{1:m}$, can be obtained.
 - Just as with encoders, decoders can be realized by any kind of sequence architecture (RNNs, LSTMs, transformers).



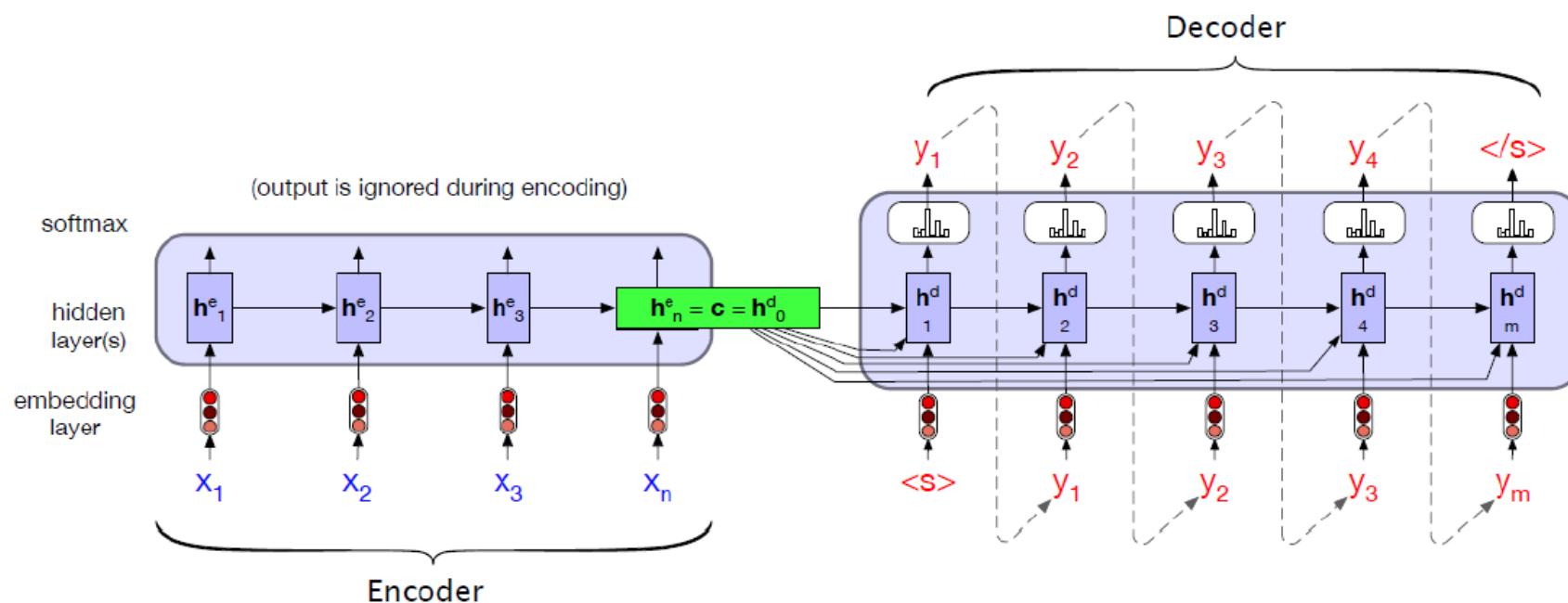
Encoder-Decoder Model with RNNs

- Translating a single sentence in the **basic RNN version of encoder-decoder approach to machine translation**.
 - Source and target sentences are concatenated with a separator token in between, and the decoder uses context information from the encoder's last hidden state.



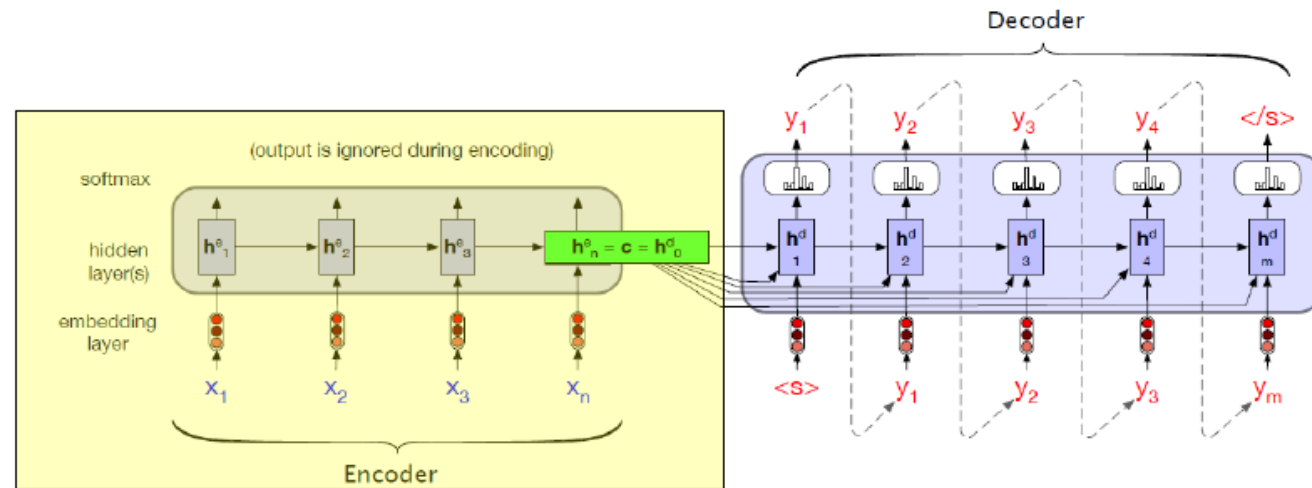
Encoder-Decoder Model with RNNs

- A formal version of translating a sentence in the basic RNN-based encoder-decoder architecture:
 - The **final hidden state** of the encoder RNN $\mathbf{h}_n^e$, serves as the **context** $\mathbf{c}$ for the decoder in its role as $\mathbf{h}_0^d$ in the decoder RNN, and
 - The **context** is also made available to each decoder hidden state.



Encoder-Decoder Model with RNNs

basic RNN-based encoder-decoder architecture



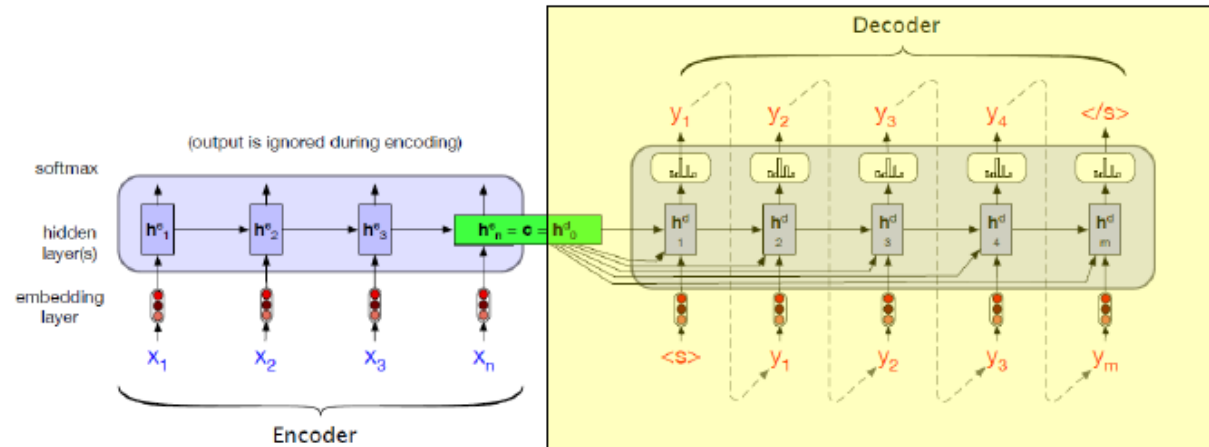
- The purpose of the **encoder** is to generate a contextualized representation of the input.
- This representation is embodied in the *final hidden state of the encoder* h^e_n , also called **c** for **context**, and it is then passed to the decoder.
- Encoder can be a single network layer,
 - Stacked architectures (such as stacked biLSTMs) are widely used to represent encoders.

Encoder-Decoder Model with RNNs

basic RNN-based encoder-decoder architecture

- Equations for the decoder, with context available at each decoding timestep.

$$\begin{aligned} \mathbf{c} &= \mathbf{h}_n^e \\ \mathbf{h}_0^d &= \mathbf{c} \\ \mathbf{h}_t^d &= g(\hat{y}_{t-1}, \mathbf{h}_{t-1}^d, \mathbf{c}) \\ \mathbf{z}_t &= f(\mathbf{h}_t^d) \\ y_t &= \text{softmax}(\mathbf{z}_t) \end{aligned}$$

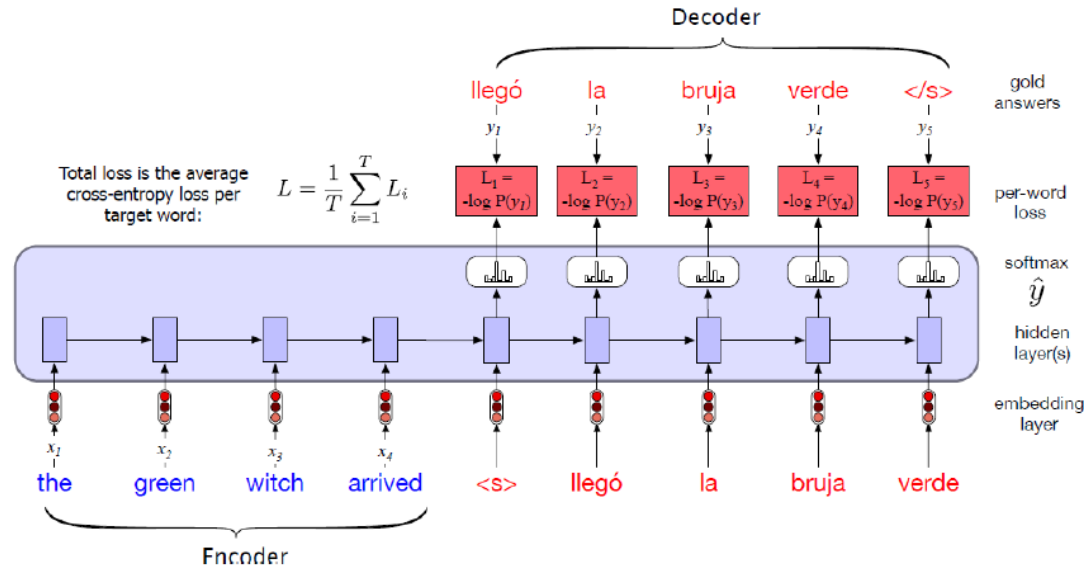


- The output y at each time step consists of a softmax computation over the set of possible outputs (the vocabulary).
- We compute the most likely output at each time step by taking the argmax over the softmax output:

$$\hat{y}_t = \operatorname{argmax}_{w \in V} P(w | y_1 \dots y_{t-1}, x)$$

Encoder-Decoder Model with RNNs

basic RNN-based encoder-decoder architecture - Training



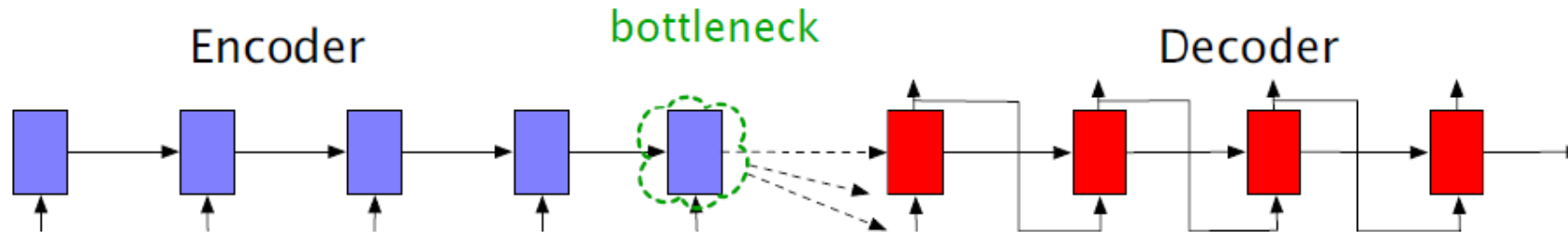
- **Training the basic RNN encoder-decoder approach to machine translation.**
 - Note that in the decoder we usually don't propagate the model's softmax outputs $\hat{y}_t$, but use **teacher forcing** to force each input to the correct gold value for training.
 - We compute the softmax output distribution over $\hat{y}$ in the decoder in order to compute the loss at each token, which can then be averaged to compute a loss for the sentence.

Encoder-Decoder Model with RNNs

- *Attention*

Encoder-Decoder Model with RNNs - Attention

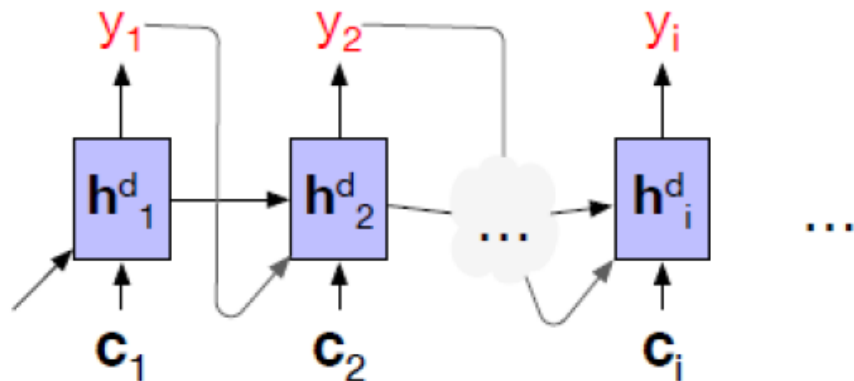
- Requiring the context c to be only the encoder's final hidden state forces all the information from the entire source sentence to pass through this representational **bottleneck**.



- The **attention mechanism** is a solution to the bottleneck problem,
- The **attention mechanism** allows the decoder to get information from all the hidden states of the encoder, not just the last hidden state.

Encoder-Decoder Model with RNNs - Attention

- The idea of attention is to create the single fixed-length vector c by taking a weighted sum of all the encoder hidden states.
- The weights focus on (**‘attend to’**) a particular part of the source text that is relevant for the token the decoder is currently producing.
- **Attention** thus replaces the static context vector with one that is dynamically derived from the encoder hidden states, different for each token in decoding.



The **attention mechanism** allows each hidden state of the decoder to see a different, dynamic, context, which is a function of all the encoder hidden states.

Encoder-Decoder Model with RNNs - Attention

- How **relevant** each encoder state is to the decoder state.
 - Similarity of j^{th} encoder hidden state $\mathbf{h}_j^e$ with $i-1^{\text{th}}$ decoder hidden state $\mathbf{h}_{i-1}^d$: $\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e)$
- Simple score function is **dot-product attention**:

$$\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e) = \mathbf{h}_{i-1}^d \cdot \mathbf{h}_j^e$$

- To use these scores, we create a **vector of weights** α_{ij} , that tells us the proportional relevance of each encoder hidden state j to the prior hidden decoder state, $\mathbf{h}_{i-1}^d$.

$$\alpha_{ij} = \text{softmax}(\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e)) = \frac{\exp(\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_j^e))}{\sum_k \exp(\text{score}(\mathbf{h}_{i-1}^d, \mathbf{h}_k^e))}$$

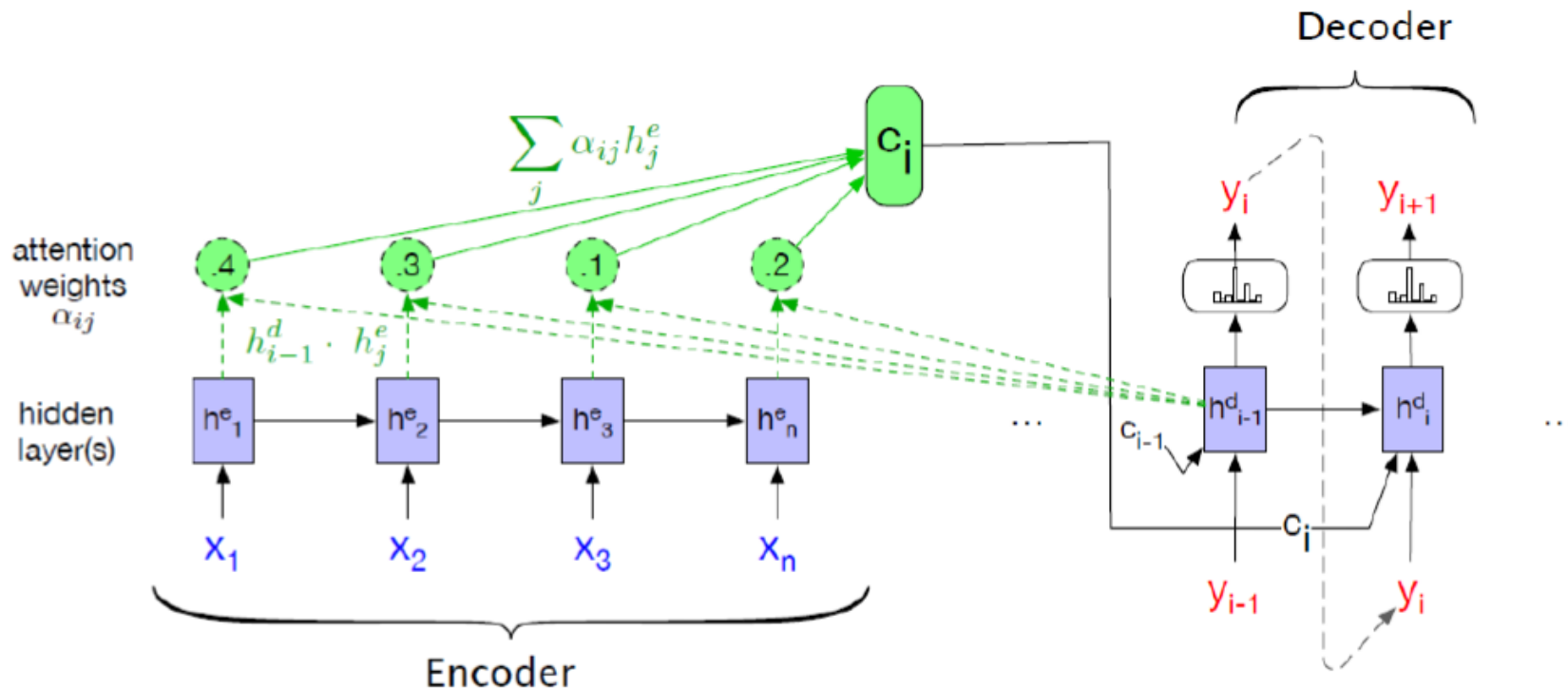
- We can compute a **fixed-length context vector for the current decoder state** by taking a weighted average over all the encoder hidden states.

$$\mathbf{c}_i = \sum_j \alpha_{ij} \mathbf{h}_j^e$$

Encoder-Decoder Model with RNNs - Attention

The encoder-decoder network with **attention**, focusing on the computation of c_i :

- The context value c_i is one of the inputs to the computation of h_i^d
- It is computed by taking the weighted sum of all the encoder hidden states, each weighted by their dot product with the prior decoder hidden state h_{i-1}^d .



Recurrent Neural Networks (RNNs) - Summary

Recurrent Neural Networks (RNNs) - Summary

- In simple Recurrent Neural Networks sequences are processed one element at a time, with the output of each neural unit at time t based both on the current input at t and the hidden layer from time $t-1$.
- RNNs can be trained with a straightforward extension of the backpropagation algorithm, known as **backpropagation through time (BPTT)**.
- Simple recurrent networks fail on long inputs because of problems like **vanishing gradients**; modern systems use more complex gated architectures such as **LSTMs** that explicitly decide what to remember and forget in their hidden and context layers.

Recurrent Neural Networks (RNNs) - Summary

Common language-based applications for RNNs include:

- **Probabilistic language modeling:** assigning a probability to a sequence, or to the next element of a sequence given the preceding words.
- **Auto-regressive generation** using a trained language model.
- **Sequence labeling** like part-of-speech tagging, where each element of a sequence is assigned a label.
- **Sequence classification**, where an entire text is assigned to a category, as in spam detection, sentiment analysis or topic classification.
- **Encoder-decoder architectures**, where an input is mapped to an output of different length and alignment.